

Getting Started with SLURM and Python Environments on an HPC Cluster

Md Shahriar Forhad
shahriar.forhad.eee@gmail.com
<https://shahriar88.github.io/>



August 15, 2026

Contents

1	Introduction to CRADLE	7
1.1	Introduction	7
1.1.1	Logging In to CRADLE	7
1.2	A Minimal SLURM Job Script Example	8
1.3	Understanding Key #SBATCH Directives	8
1.3.1	Number of Nodes: <code>-N</code>	9
1.3.2	Partition (Queue): <code>-p</code>	9
1.3.3	GPU Request: <code>-gres</code>	9
1.3.4	CPU Cores per Task: <code>-cpus-per-task</code>	9
1.3.5	Time Limit: <code>-time</code>	10
1.3.6	Summary Table of Core Directives	10
1.4	Why Use Python Environments on an HPC Cluster?	10
2	CRADLE COMMAND	13
2.1	Transferring Files to and from the CRADLE Cluster	13
2.1.1	Using <code>scp</code> (Secure Copy)	13
2.1.2	Working with Directories in the Terminal	14
2.1.3	Cloning a GitHub Repository	14
2.1.4	Recommended Best Practices	14
2.2	Cloning a Shared TensorFlow Environment	15
2.2.1	Step 1: Install the cloning tool (one time)	15
2.2.2	Step 2: Clone the shared environment	15
2.2.3	Step 3: Activate the cloned environment	16
2.2.4	Step 4: Add new packages (example: <code>pandas</code>)	16
2.2.5	Step 5: Use the cloned environment in your SLURM job	16
2.2.6	Mental Model Summary	17
2.3	Creating Your Own Python Virtual Environment	17
2.3.1	Option A (Recommended for CPU-Only): Create from Scratch	17
2.3.2	Option B (Best for GPU): Clone an Optimized Shared Environment	18
2.4	Verifying Your Environment and Common Pitfalls	18
2.4.1	Verifying Python and GPU Visibility	18
2.4.2	Common Mistakes to Avoid	19
2.4.3	Quick Decision Guide	19
2.5	Using a PyTorch Environment on CRADLE	19
2.5.1	Minimal GPU Verification Checklist	19
2.6	Using Micromamba and Portable Conda Environments on the HPC Cluster	20
2.6.1	Exporting a Portable Conda Environment on Your PC	20

2.6.2	Copying the YAML File to CRADLE	20
2.6.3	Installing Micromamba in Your Home Directory	21
2.6.4	Creating a Micromamba Environment from the YAML File	21
2.6.5	Activating the Environment on CRADLE	21
2.6.6	Using a Micromamba Environment in a SLURM Job	22
2.6.7	Verifying GPU Support Inside the Environment	23
2.6.8	Best Practices and Common Pitfalls for Micromamba on HPC	23
2.7	Worked Examples: Environments and SLURM Job Scripts	23
2.7.1	Creating a New CPU-Only Environment	24
2.7.2	Cloning a Shared GPU-Optimized TensorFlow Environment	24
2.7.3	Selecting an Environment in a Job Script	24
2.7.4	Installing Libraries After Environment Activation	25
2.7.5	Checking and Verifying Resources	25
2.8	PyTorch DataParallel (DP) on SLURM	28
2.8.1	When to Use DataParallel	28
2.8.2	When <i>Not</i> to Use DataParallel	28
2.8.3	Minimal DataParallel Training Skeleton	29
2.8.4	SLURM Script for DataParallel (single node)	31
2.8.5	DP vs DDP: Practical Comparison	31
2.8.6	Common DP Pitfalls	31
2.8.7	Rule of Thumb	32
2.9	PyTorch Distributed Data Parallel (DDP) on SLURM	32
2.9.1	What You Need (Checklist)	32
2.9.2	Minimal DDP Training Skeleton (single file)	32
2.9.3	SLURM Script for DDP (single node, 4 GPUs)	34
2.9.4	SLURM Script for DDP (2 nodes, 4 GPUs per node)	34
2.9.5	Common DDP Pitfalls (Fast Debug List)	35
3	Example Commands	37
3.1	Example Commands for File Transfer	37
3.1.1	Copy files from/to CRADLE	41
3.1.2	Installing Anaconda or Miniconda on CRADLE	43
3.1.3	Example SLURM Job Script Using a Conda Environment	44
3.1.4	Sample Run code for sh	46
3.1.5	Launching Jupyter Notebook on CRADLE	51
3.1.6	Troubleshooting Deleting a Folder	52
3.2	Checking Previous Slurm Job Submissions	53
3.2.1	Single Node, CPU-Only Job	53
3.2.2	Quick Mapping: Which Example to Use?	57
3.3	Summary	57
4	Deploying PyTorch Models with ONNX, Pygame, and Standalone Executables	59
4.1	Introduction	59
4.2	Meaning of Deployment Without PyTorch	59
4.2.1	Alternative Deployment Options	60
4.3	Step-by-Step Deployment Plan	60
4.4	Recommended Project Structure	61
4.5	Creating and Exporting the Model	61

4.5.1	Example Model Definition	61
4.5.2	Loading the Checkpoint	62
4.5.3	Exporting to ONNX	63
4.5.4	Checking Numerical Agreement	63
4.6	Designing the Pygame Inference Application	64
4.6.1	Configuration File	64
4.6.2	Locating Bundled Files	65
4.6.3	Loading ONNX Runtime	65
4.6.4	Preprocessing, Inference, and Postprocessing	66
4.6.5	Displaying a PIL Image in Pygame	68
4.6.6	Keeping the Interface Responsive	68
4.7	Installing the Development Dependencies	70
4.8	Building a Standalone Runtime	70
4.8.1	Windows Build	70
4.8.2	Linux Build	72
4.8.3	Operating-System Compatibility	72
4.9	CPU and GPU Deployment	72
4.10	Testing the Final Application	72
4.10.1	Common Problems	73
4.11	Adapting the Example to a Trained Model	73
4.12	Summary	74
5	Publishing an ML Demo with GitHub Pages and an On-Demand Inference API	75
5.1	Introduction	75
5.2	Understanding What Loads and When	76
5.3	Browser Inference Versus API Inference	76
5.4	End-to-End Request Sequence	77
5.5	Recommended Project Organization	77
5.6	Building the GitHub Pages Frontend	78
5.6.1	HTML Interface	78
5.6.2	Basic Styling	79
5.6.3	Previewing the Image Without Calling the API	80
5.6.4	Calling the API Only After a Button Click	81
5.6.5	Adding the Demo as a Separate Page	83
5.7	Building the FastAPI Inference Service	84
5.7.1	Loading the Model Once	84
5.7.2	Preprocessing and Prediction	84
5.7.3	Creating the API Application	85
5.7.4	Response Formats	87
5.8	Running the API Locally	87
5.9	Containerizing the API	89
5.10	Choosing an API Hosting Pattern	90
5.11	Configuring CORS Correctly	90
5.12	Security, Privacy, and Abuse Prevention	92
5.12.1	Do Not Put a Secret API Key in Public JavaScript	92
5.12.2	Uploaded Image Privacy	93
5.13	Managing Latency and User Experience	93
5.13.1	Preventing Duplicate Requests	93

5.14	Scaling and Concurrency	94
5.15	Versioning the Model and API	94
5.16	Testing Before Public Release	95
5.16.1	Common Problems	95
5.17	Deployment Checklist	95
5.18	Summary	96

Chapter 1

Introduction to CRADLE

1.1 Introduction

For more details <https://www.geeksforgeeks.org/linux-unix/scp-command-in-linux-with-examples/>

This short tutorial explains how to:

- Write a basic SLURM batch script for running a TensorFlow job.
- Understand key `#SBATCH` directives, including nodes, partitions, GPUs, and time limits.
- Use and extend a shared TensorFlow environment on the cluster.
- Create your own Python virtual environments for CPU and GPU workloads.

All examples are designed to be copied directly into a terminal or SLURM job script on a typical HPC system (e.g., a university cluster).

1.1.1 Logging In to CRADLE

Once you have received your user account information, you can log in to the CRADLE cluster from a terminal or command-line prompt using your University credentials.

Step 1: Connect to the University Network (VPN)

If you are off campus, you must first connect to the University network using the approved VPN software. Install and start the VPN client, then log in with your University credentials. Once the VPN connection is active, you can reach the CRADLE login node.

Step 2: SSH into the Cluster

To log in, use `ssh` with your assigned username. In the examples below, replace `username` with your actual cluster username.

```
1 ssh username@login.cradle.university.edu
```

You will be prompted for your password:

```
1 username@login.cradle.university.edu's password:
```

After you enter your password correctly, you should see a prompt indicating that you are now on the cluster login node, similar to:

```
1 [username@login001 ~]\$
```

At this point you are logged into CRADLE and ready to work with SLURM, Python environments, and the other tools described in this tutorial.

1.2 A Minimal SLURM Job Script Example

Listing 1.1 shows a basic SLURM job script that runs a TensorFlow test script in a shared environment.

Listing 1.1: Basic SLURM job script with TensorFlow environment

```
1 #!/bin/bash
2
3 # Job identification
4 #SBATCH --job-name=myFirstJob
5
6 # Standard output and error files (%j = job ID)
7 #SBATCH --output=myFirstJob.out.%j
8 #SBATCH --error=myFirstJob.err.%j
9
10 # Resource requests
11 #SBATCH -N 1
12 #SBATCH -p kimg
13 #SBATCH --gres=gpu:1
14 #SBATCH --time=1:00:00
15
16 # Display GPU assigned by SLURM
17 echo "CUDA_VISIBLE_DEVICES: ${CUDA_VISIBLE_DEVICES}"
18
19 # Activate TensorFlow virtual environment
20 echo "Activating TensorFlow-2.6.2 environment"
21 source /shared/tensorflow-2.6.2/tf_env/bin/activate
22
23 # Run TensorFlow test script
24 echo "Running testTF.py"
25 python3 ~/testTFForSlurm/testTF.py
26
27 # Deactivate environment
28 echo "Deactivating TensorFlow-2.6.2 environment"
29 deactivate
```

To submit this job to SLURM, save the script as `myFirstJob.slurm` (for example) and run:

```
1 sbatch myFirstJob.slurm
```

1.3 Understanding Key #SBATCH Directives

The #SBATCH lines at the top of the script tell SLURM what resources you want. They are not shell commands; they are scheduler directives.

1.3.1 Number of Nodes: `-N`

```
1 #SBATCH -N 1
```

This requests **1 compute node**. A node is a physical machine that may contain many CPU cores, memory, and possibly GPUs.

More examples:

```
1 #SBATCH -N 1 # Single-node TensorFlow training
2 #SBATCH -N 2 # Distributed job across 2 nodes
3 #SBATCH -N 1-4 # SLURM may allocate anywhere from 1 to 4 nodes
```

1.3.2 Partition (Queue): `-p`

```
1 #SBATCH -p kimq
```

The partition selects the *group of nodes* (queue) your job will run on.

On CRADLE, the GPU partition used in the examples below is `gpua30q`. Always verify available partitions with `sinfo`, since partition names are cluster-specific.

Typical partitions might be:

- `compute` → Standard CPU nodes
- `bigmem` → High-memory nodes
- `gpu` or `gpua30q` → GPU-enabled nodes

Examples:

```
1 #SBATCH -p kimq # Run on GPU nodes in the kimq partition
2 #SBATCH -p compute # Run on CPU-only nodes
```

1.3.3 GPU Request: `-gres`

```
1 #SBATCH --gres=gpu:1
```

This requests **1 GPU** using the generic resource (GRES) mechanism.

More examples:

```
1 #SBATCH --gres=gpu:1 # One GPU
2 #SBATCH --gres=gpu:2 # Two GPUs
3 #SBATCH --gres=gpu:a100:1 # One NVIDIA A100 GPU (if supported)
```

Important: Partition and GPU must match. If you request GPUs but choose a CPU-only partition (e.g., `-p compute` with `-gres=gpu:1`), SLURM cannot satisfy the request and the job will stay pending or fail.

1.3.4 CPU Cores per Task: `-cpus-per-task`

```
1 #SBATCH --cpus-per-task=2
```

This requests CPU cores for the main process. In GPU jobs, these CPU cores are commonly used for data loading, preprocessing, and other host-side tasks. If your Python code uses `DataLoader(num_workers > 0)`, you usually need to request enough CPU cores to support it.

More examples:

```
1 #SBATCH --cpus-per-task=2 # Light data loading
2 #SBATCH --cpus-per-task=4 # Moderate preprocessing
3 #SBATCH --cpus-per-task=8 # Heavier data loading or multi-worker pipeline
```

1.3.5 Time Limit: -time

```
1 #SBATCH --time=1:00:00
```

This sets a **wall-clock time limit** of 1 hour for your job. When this time is reached, SLURM terminates the job.

More examples:

```
1 #SBATCH --time=00:30:00 # 30 minutes
2 #SBATCH --time=04:00:00 # 4 hours
3 #SBATCH --time=2-00:00:00 # 2 days
```

1.3.6 Summary Table of Core Directives

1.4 Why Use Python Environments on an HPC Cluster?

On HPC clusters, shared software environments (for example, a shared TensorFlow installation) are usually:

- Optimized for the cluster hardware (correct CUDA, cuDNN, drivers).
- Read-only for users.
- Not modifiable by individual users.

You *cannot* safely run:

```
1 pip install pandas
```

inside a shared environment such as

```
1 /shared/tensorflow-2.6.2/tf_env/
```

because:

- It is shared by everyone.
- You do not have write permission.
- Changes could break other users' jobs.

Solution: Clone the shared environment into your home directory, then customize the copy.

Command	Description	Example
Basic Linux Commands		
pwd	Print current working directory.	pwd
ls	List files in current directory.	ls
ls -l	List files with detailed information.	ls -l
cd <dir>	Change directory.	cd projects
mkdir -p <dir>	Create directory (including parent directories if needed).	mkdir -p data/images
cp <src> <dst>	Copy file or directory.	cp file.txt backup/
mv <src> <dst>	Move or rename file.	mv old.txt new.txt
rm <file>	Remove file.	rm temp.txt
rm -rf <file>	Remove file by force	rm -rf temp.txt
clear	Clear the terminal screen.	clear
history	Show previously executed commands.	history
whoami	Show current username.	whoami
hostname	Show the current machine/node name.	hostname
Account and Security		
passwd	Change your login password.	passwd
SLURM / HPC Commands		
sinfo	Show available partitions and node status.	sinfo
squeue	Show all running jobs.	squeue
squeue -u <user>	Show jobs for specific user.	squeue -u username
sbatch <script.sh>	Submit batch job.	sbatch train.slurm
srunch -pty bash	Start interactive session on compute node.	srunch -pty bash
scancel <jobid>	Cancel a running job.	scancel 12345
scontrol show job <jobid>	Display detailed information about a job.	scontrol show job 12345
GPU and Monitoring		
nvidia-smi	Display GPU usage (compute nodes only).	nvidia-smi
tail -f <file>	Monitor job output in real time.	tail -f job.out
Environment and Modules		
module avail	List available software modules.	module avail
module load <name>	Load a software module.	module load cuda/12.3
module list	Show currently loaded modules.	module list
which python	Show the path of the active Python interpreter.	which python
Storage and Usage		
du -sh *	Show directory sizes.	du -sh *
df -h	Show disk usage.	df -h
quota -s	Show storage quota.	quota -s
nano <file>	Edit files.	nano scheduler.sh
less <file>	View large files interactively.	less output.log

Table 1.1: Common Linux and SLURM commands for CRADLE HPC usage

Directive	Controls	Example
<code>-N</code>	Number of nodes	<code>-N 1</code>
<code>-p</code>	Hardware pool	<code>-p gpua30q</code>
<code>-gres</code>	GPU count	<code>gpu:1</code>
<code>-cpus-per-task</code>	CPU cores per task	<code>2</code>
<code>-time</code>	Runtime limit	<code>1:00:00</code>

Table 1.2: Core SLURM directives and examples.

Chapter 2

CRADLE COMMAND

2.1 Transferring Files to and from the CRADLE Cluster

When working on the CRADLE HPC cluster, you will often need to move files between your local computer and the cluster. Common examples include:

- Uploading scripts, data files, or configuration files to the cluster.
- Downloading results, logs, or trained models back to your local machine.

This section presents two recommended and commonly used methods:

1. Secure Copy (`scp`)
2. Cloning repositories from GitHub

2.1.1 Using `scp` (Secure Copy)

The `scp` command allows you to securely transfer files between your local machine and the cluster over SSH. It works on Linux, macOS, and Windows (with PowerShell, Git Bash, or WSL).

Transferring a Local File to the Cluster

To copy a file from your local directory to your home directory on the cluster:

```
1 scp localFileInMyDirectory.txt \  
2   yourUserName@login.cradle.university.edu:~/destinationForYourFile/
```

Explanation:

- `localFileInMyDirectory.txt` is the file on your local computer.
- `yourUserName@login.cradle.university.edu` is your cluster login.
- `~/destinationForYourFile/` is the target directory on the cluster.

If the destination directory does not exist, create it first after logging in:

```
1 mkdir -p ~/destinationForYourFil
```

Transferring a File from the Cluster to Your Local Machine

To copy a file from the cluster back to your local directory:

```
1 scp yourUserName@login.cradle.university.edu:~/destinationForYourFile/
   localFileInMyDirectory.txt \
2 .
```

Here, the dot (.) represents your current local directory.

2.1.2 Working with Directories in the Terminal

In both `scp` examples above, the file name refers to a file located in your *current working directory*. You can usually identify this directory by looking at your terminal prompt.

For example: `1 [username@mycomputer:~/Desktop/test]$`

This indicates that your current directory is: `1 ~/Desktop/test`

You can explicitly check your current directory using:

On Linux or macOS: `1 pwd`

This prints something like: `1 /home/username/Desktop/test`

On Windows (PowerShell): `1 cd`

Understanding your current directory is essential to avoid copying the wrong files or encountering file not found errors.

2.1.3 Cloning a GitHub Repository

Instead of manually transferring files, it is often easier to store your project code in a GitHub repository and clone it directly on the cluster.

Example: Cloning a Repository

After logging into the cluster, run:

```
1 git clone https://github.com/emenriquez/testTFForSlurm
```

This creates a new directory called `testTFForSlurm` containing the full repository.

Advantages of using GitHub:

- Version control and change tracking.
- Easy synchronization across multiple machines.
- Clean separation between code and large datasets.

2.1.4 Recommended Best Practices

- Use `scp` for small files, configuration files, and quick transfers.
- Use GitHub for project code and scripts.

- Avoid transferring very large datasets via `scp`; instead, place them directly on shared cluster storage if available.
- Keep your home directory organized (e.g., `projects/`, `data/`, `results/`).

These practices help ensure efficient workflows and reduce errors when working between your local system and the CRADLE cluster.

2.2 Cloning a Shared TensorFlow Environment

2.2.1 Step 1: Install the cloning tool (one time)

```
1 pip3 install --user virtualenv-clone
```

This:

- Installs `virtualenv-clone` into your user space.
- Does not require admin/root access.
- Only needs to be run once.

2.2.2 Step 2: Clone the shared environment

```
1 virtualenv-clone /shared/tensorflow-2.6.2/tf_env ~/myEnvs/tf_env
```

Here:

- `/shared/tensorflow-2.6.2/tf_env/` is the original, optimized, read-only TensorFlow environment.
- `~/myEnvs/tf_env` is your private, writable copy.

What gets copied:

- Python version.
- TensorFlow build (CUDA, cuDNN matched to cluster GPUs).
- All installed dependencies.
- Environment configuration tuned for the cluster.

Advantages:

- Runs as fast as the shared environment.
- You can safely add or upgrade packages.
- No conflicts with other users.

2.2.3 Step 3: Activate the cloned environment

```
1 source ~/myEnvs/tf_env/bin/activate
```

After activation:

- `python` and `pip` point to your cloned environment.
- Any `pip install` affects only your copy.

The shell prompt typically changes to something like:

```
1 (tf_env) username@node \
```

2.2.4 Step 4: Add new packages (example: pandas)

With the environment active:

```
1 pip3 install pandas
```

What happens:

- `pandas` is installed only into `~/myEnvs/tf_env`.
- TensorFlow and CUDA support remain intact.
- The customization is isolated and safe.

You can also add:

```
1 pip install scikit-learn matplotlib seaborn tqdm
```

2.2.5 Step 5: Use the cloned environment in your SLURM job

Update the activation lines in your job script.

Old (shared environment):

```
1 echo "Activating TensorFlow-2.6.2 environment"
2 source /shared/tensorflow-2.6.2/tf_env/bin/activate
```

New (your cloned environment):

```
1 echo "Activating my custom environment"
2 source ~/myEnvs/tf_env/bin/activate
```

This ensures:

- Your job sees TensorFlow and any extra packages (e.g., `pandas`).
- You avoid dependency errors at runtime.
- The behavior of your job is reproducible.

Piece	Purpose
Shared environment	Fast, optimized, read-only
Cloned environment	Same speed, user-modifiable
<code>virtualenv-clone</code>	Copies entire environment
<code>pip install</code>	Safe only in cloned environment
Job script	Must activate your chosen environment

Table 2.1: Conceptual roles of shared and cloned environments.

2.2.6 Mental Model Summary

2.3 Creating Your Own Python Virtual Environment

Sometimes you want a completely independent Python environment. There are two main options; here we focus on standard `venv` and the cloned approach.

2.3.1 Option A (Recommended for CPU-Only): Create from Scratch

Step 1: Choose a location

Create a directory to hold your environments:

```
1 mkdir -p ~/myEnvs
```

Step 2: Create the virtual environment

```
1 python3 -m venv ~/myEnvs/myenv
```

This creates something like:

```
1 ~/myEnvs/myenv/
2 |- bin/
3 |- lib/
4 '- pyvenv.cfg
```

Step 3: Activate the environment

```
1 source ~/myEnvs/myenv/bin/activate
```

Your shell prompt changes to:

```
1 (myenv) username@login $
```

Step 4: Upgrade pip (recommended)

```
1 pip install --upgrade pip
```

Step 5: Install packages

Example:

```
1 pip install tensorflow pandas numpy matplotlib
```

Note: On clusters, installing TensorFlow this way may not include GPU support unless CUDA and drivers are correctly matched. For GPU workloads, the cloned shared environment (Option B) is often safer.

Step 6: Use this environment in a SLURM job

In your job script:

```
1 source ~/myEnvs/myenv/bin/activate
2 python train.py
```

2.3.2 Option B (Best for GPU): Clone an Optimized Shared Environment

This is essentially the workflow described in Section 2.2.

Step 1: Install the clone tool:

```
1 pip install --user virtualenv-clone
```

Step 2: Clone the shared environment:

```
1 virtualenv-clone /shared/tensorflow-2.6.2/tf_env ~/myEnvs/tf_env
```

Step 3: Activate it:

```
1 source ~/myEnvs/tf_env/bin/activate
```

Step 4: Add packages:

```
1 pip install pandas scikit-learn seaborn
```

Step 5: Update your SLURM job to activate:

```
1 source ~/myEnvs/tf_env/bin/activate
```

This preserves CUDA/cuDNN compatibility and performance while allowing you to customize the environment.

2.4 Verifying Your Environment and Common Pitfalls

2.4.1 Verifying Python and GPU Visibility

After activating an environment, verify what `python` and `pip` you are using:

```
1 which python
2 which pip
```

For TensorFlow GPU detection:

```
1 python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

Optional but often required: load CUDA module Some clusters require loading a CUDA module before running GPU-enabled PyTorch or TensorFlow jobs. For example:

```
1 module load cuda/12.3
```

follow the cluster documentation or the instructions provided by your system administrator.

You should see at least one GPU listed when running on a GPU node with correct drivers and environment.

2.4.2 Common Mistakes to Avoid

- Installing packages without activating the environment first.
- Using `pip install -user` inside a virtual environment.
- Mixing Conda environments and `venv` in the same workflow.
- Using the system Python (no virtual environment) for SLURM jobs.
- Requesting GPUs in a CPU-only partition.

2.4.3 Quick Decision Guide

Situation	Best choice
CPU-only work	Option A (new <code>venv</code>)
GPU + TensorFlow	Option B (clone shared env)
Cluster-optimized builds	Option B
Full custom control	Option A

Table 2.2: Choosing between creating and cloning environments.

2.5 Using a PyTorch Environment on CRADLE

If your cluster provides a shared PyTorch environment, activate it in your SLURM script using the path provided by your instructor, system administrator, or cluster documentation.

```
1 source /path/to/shared/pytorch/environment/bin/activate
```

You may also use your own cloned or user-created environment if it includes a CUDA-compatible PyTorch installation.

Then verify that PyTorch can see the GPU:

```
1 python3 - << 'PY'
2 import os, torch
3 print("Torch:", torch.__version__)
4 print("CUDA available:", torch.cuda.is_available())
5 print("Device count:", torch.cuda.device_count())
6 print("CUDA_VISIBLE_DEVICES:", os.environ.get("CUDA_VISIBLE_DEVICES"))
7 if torch.cuda.device_count() > 0:
8     for i in range(torch.cuda.device_count()):
9         print(f"GPU {i}:", torch.cuda.get_device_name(i))
10 PY
```

This check confirms that the active Python environment contains a CUDA-enabled PyTorch build and that the GPU allocated by SLURM is visible inside the job.

2.5.1 Minimal GPU Verification Checklist

For a GPU job, verify the following inside the SLURM script:

1. `echo $CUDA_VISIBLE_DEVICES`

2. `nvidia-smi`
3. Framework-level check such as `torch.cuda.is_available()` or TensorFlow GPU detection

2.6 Using Micromamba and Portable Conda Environments on the HPC Cluster

On many HPC clusters, including CRADLE, traditional Conda/Anaconda installations are discouraged or restricted because they are large, modify shell startup files aggressively, and may conflict with system libraries or CUDA drivers. A lightweight and HPC-friendly alternative is *Micromamba*, a single static binary that implements the Conda ecosystem without requiring a global base environment.

Micromamba allows you to:

- Recreate a Conda environment from a portable YAML file exported on your laptop.
- Install packages into your home directory without administrator rights.
- Activate environments explicitly inside SLURM job scripts.

The overall workflow is:

1. Export a portable Conda environment YAML on your local machine.
2. Copy the YAML file to CRADLE.
3. Install Micromamba in your home directory on CRADLE.
4. Create a Micromamba environment from the YAML file.
5. Activate the environment in your SLURM job scripts.

2.6.1 Exporting a Portable Conda Environment on Your PC

On your local machine where you already use Conda, start by creating a portable YAML description of your environment. Suppose your local environment is called `IR`.

First, activate it:

```
1 conda activate IR
```

Then export a *portable* environment file using `fromhistory`:

```
1 conda env export --from-history > IR_portable.yml
```

This creates a file `IR_portable.yml` containing the channels and high-level dependencies (without build hashes). It is much more likely to recreate successfully on a different system such as CRADLE.

2.6.2 Copying the YAML File to CRADLE

Use `scp` or an equivalent file transfer method to copy `IR_portable.yml` to your home directory on CRADLE. For example, from your local terminal:

```
1 scp IR_portable.yml username@login.cradle.university.edu:~
```

After logging into CRADLE, you should see the file in your home directory:

```
1 ls ~/IR_portable.yml
```

2.6.3 Installing Micromamba in Your Home Directory

Micromamba is distributed as a single binary, which makes it ideal for installation in a user home directory (no root access required).

Create a directory for the binary:

```
1 mkdir -p ~/bin
2 cd ~/bin
```

Download and unpack the latest Micromamba binary (Linux 64-bit):

```
1 curl -L https://micro.mamba.pm/api/micromamba/linux-64/latest \
2 | tar -xvj bin/micromamba
```

Add `$HOME/bin` to your PATH in `~/.bashrc`:

```
1 echo 'export PATH=$HOME/bin:$PATH' >> ~/.bashrc
2 source ~/.bashrc
```

Verify that `micromamba` is available:

```
1 micromamba --version
```

2.6.4 Creating a Micromamba Environment from the YAML File

Choose a directory where Micromamba will store its environments, for example:

```
1 mkdir -p ~/micromamba/envs
```

Initialize Micromamba for Bash (one time):

```
1 micromamba shell init -s bash -p ~/micromamba
2 source ~/.bashrc
```

Now create a new environment (for example, `IR_NEW`) from your portable YAML file:

```
1 micromamba create \
2 -f ~/IR_portable.yml \
3 -n IR_NEW \
4 -p ~/micromamba/envs/IR_NEW
```

Here:

- `~/IR_portable.yml` is the YAML exported on your PC.
- `IR_NEW` is the environment name.
- `~/micromamba/envs/IR_NEW` is the full path Micromamba uses.

2.6.5 Activating the Environment on CRADLE

Once created, you can activate the environment on the CRADLE login node:

```
1 micromamba activate ~/micromamba/envs/IR_NEW
```

After activation:

- `python` and `pip` refer to the Micromamba environment.
- Any installs via `pip` or `micromamba install` affect only that environment.

You can verify:

```
1 which python
2 python --version
3 pip list
```

2.6.6 Using a Micromamba Environment in a SLURM Job

In a SLURM job script, you should explicitly set the Micromamba root and activate the environment before running Python code. For example:

```
1  #!/bin/bash
2  #SBATCH --job-name=ir_gpu_job
3  #SBATCH --output=ir_gpu_job.out.%j
4  #SBATCH --error=ir_gpu_job.err.%j
5  #SBATCH -N 1
6  #SBATCH -p kimq # GPU partition
7  #SBATCH --gres=gpu:1 # 1 GPU
8  #SBATCH --time=02:00:00 # 2 hours
9
10 # Ensure Micromamba root is defined
11 export MAMBA_ROOT_PREFIX=$HOME/micromamba
12
13 # Load shell initialization (if micromamba shell init was used)
14 source $HOME/.bashrc
15
16 echo "Host: $(hostname)"
17 echo "CUDA_VISIBLE_DEVICES: $CUDA_VISIBLE_DEVICES"
18
19 echo "Activating Micromamba environment IR_NEW"
20 micromamba activate $HOME/micromamba/envs/IR_NEW
21
22 echo "Checking TensorFlow GPU visibility"
23 python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
24
25 echo "Running training script"
26 python ~/projects/train_ir_model.py
```

This pattern ensures that:

- The correct Micromamba environment is active inside the job.
- GPU visibility can be checked via TensorFlow (or PyTorch).
- The job is reproducible and does not depend on the login-node Conda setup.

2.6.7 Verifying GPU Support Inside the Environment

To confirm that your Micromamba environment has working GPU support for TensorFlow (or another framework), you can run:

```
1 python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

On a GPU node with correct drivers and CUDA libraries, this should list one or more GPU devices. You can also check the low-level GPU visibility with:

```
1 nvidia-smi
```

2.6.8 Best Practices and Common Pitfalls for Micromamba on HPC

Best practices:

- Export environments from Conda using `-from-history` or `-no-builds` to avoid hardware-specific build strings.
- Use Micromamba to recreate the environment on the cluster, not full Anaconda.
- Keep all Micromamba files under `~/micromamba` or a dedicated directory in your home space.
- Activate environments explicitly inside SLURM job scripts.

Common pitfalls:

- Installing full Anaconda on an HPC login node (disk usage and conflicts).
- Relying on Conda's `base` environment in batch jobs.
- Mixing Conda and Micromamba environments in the same workflow.
- Installing GPU frameworks with incompatible `cuda-toolkit` versions; always check which CUDA versions the cluster supports.

Using Micromamba with a portable Conda YAML exported from your local machine gives you a reproducible, HPC-safe way to manage Python environments on CRADLE while respecting cluster policies and maximizing compatibility with the installed NVIDIA drivers and CUDA libraries.

2.7 Worked Examples: Environments and SLURM Job Scripts

This section provides concrete, copy-pasteable examples covering:

1. Creating a new environment.
2. Cloning a shared TensorFlow environment.
3. Selecting (activating) an environment.
4. Installing extra libraries.
5. Checking and verifying resources.
6. Writing SLURM scripts for different resource requirements.

2.7.1 Creating a New CPU-Only Environment

```
1 # 1. Create a directory to store your environments
2 mkdir -p ~/myEnvs
3
4 # 2. Create a new CPU-only Python virtual environment
5 python3 -m venv ~/myEnvs/cpu_env
6
7 # 3. Activate the environment
8 source ~/myEnvs/cpu_env/bin/activate
9
10 # 4. Upgrade pip
11 pip install --upgrade pip
12
13 # 5. Install some common libraries
14 pip install numpy scipy pandas matplotlib scikit-learn
15
16 # 6. When finished, deactivate
17 deactivate
```

2.7.2 Cloning a Shared GPU-Optimized TensorFlow Environment

```
1 # 1. Install the clone tool (only once per user)
2 pip3 install --user virtualenv-clone
3
4 # 2. Create a directory to store your environments (if not already created)
5 mkdir -p ~/myEnvs
6
7 # 3. Clone the shared TensorFlow environment into your home directory
8 virtualenv-clone /shared/tensorflow-2.6.2/tf_env ~/myEnvs/tf_env
9
10 # 4. Activate the cloned environment
11 source ~/myEnvs/tf_env/bin/activate
12
13 # 5. Install extra libraries needed for your project
14 pip install pandas scikit-learn matplotlib seaborn tqdm
15
16 # 6. Deactivate when done
17 deactivate
```

2.7.3 Selecting an Environment in a Job Script

Activate the CPU-only environment:

```
1 # Inside your SLURM job script
2 echo "Activating CPU-only environment"
3 source ~/myEnvs/cpu_env/bin/activate
```

Activate the cloned GPU TensorFlow environment:

```
1 # Inside your SLURM job script
2 echo "Activating GPU TensorFlow environment"
3 source ~/myEnvs/tf_env/bin/activate
```


2.7.4 Installing Libraries After Environment Activation

Inside the CPU-only environment:

```
1 source ~/myEnvs/cpu_env/bin/activate
2 pip install jupyter notebook ipykernel
3 deactivate
```

Inside the cloned GPU environment:

```
1 source ~/myEnvs/tf_env/bin/activate
2 pip install opencv-python pillow tensorboard
3 deactivate
```

2.7.5 Checking and Verifying Resources

This subsection shows how to verify what resources are available on the cluster and what resources SLURM has actually allocated to your jobs.

Check Available Partitions and Nodes

To see which partitions and nodes exist and their current status:

```
1 # Show all partitions and their status
2 sinfo
```

For more detailed information about each node:

```
1 # Detailed node information
2 scontrol show nodes
```

Check Your Jobs

To see all of your jobs in the queue:

```
1 # Show jobs for your user
2 squeue -u $USER
```

To inspect a specific job by its job ID:

```
1 # Show a specific job
2 squeue -j <JOBID>
```

Inspect Resources Allocated by SLURM

To see precisely what resources were allocated to a job:

```
1 # Show full job allocation details
2 scontrol show job <JOBID>
```

Look for fields such as NumNodes, NumCPUs, Gres, and NodeList.

Check Resources Inside a Job Script

Inside a running job, you can query SLURM environment variables to see what was allocated:

```
1 echo "Job ID: $SLURM_JOB_ID"
2 echo "Nodes allocated: $SLURM_NNODES"
3 echo "Node list: $SLURM_NODELIST"
4 echo "CPUs per node: $SLURM_CPUS_ON_NODE"
5 echo "Tasks per node: $SLURM_TASKS_PER_NODE"
6 echo "Job partition: $SLURM_JOB_PARTITION"
```

Check GPU Allocation

For GPU jobs, it is important to verify which GPUs are visible:

```
1 # GPUs assigned by SLURM (logical IDs)
2 echo "CUDA_VISIBLE_DEVICES: $CUDA_VISIBLE_DEVICES"
3
4 # List visible GPUs and utilization
5 nvidia-smi
```

Check CPU and Memory Usage

You can inspect CPU and memory characteristics on the node:

```
1 # CPU info
2 lscpu
3
4 # Memory info
5 free -h
6
7 # Per-node memory summary (first lines of /proc/meminfo)
8 cat /proc/meminfo | head
```

Check Resources Used by Completed Jobs

After a job finishes, accounting information can be retrieved with `sacct`:

```
1 # Accounting info for a finished job
2 sacct -j <JOBID> --format=JobID,JobName,Partition,AllocCPUS,Elapsed,MaxRSS,State
```

This shows how many CPUs were allocated, how long the job ran, peak memory usage (`MaxRSS`), and the final state (e.g., `COMPLETED`, `FAILED`, `TIMEOUT`).

Minimal Resource Debug Job

The following SLURM script does nothing except report which resources it received. It is useful when testing new partitions or debugging allocation issues:

```

1  #!/bin/bash
2  #SBATCH --job-name=resource_debug
3  #SBATCH -N 1
4  #SBATCH -p kimg
5  #SBATCH --gres=gpu:1
6  #SBATCH --time=00:10:00
7
8  echo "Job ID: $SLURM_JOB_ID"
9  echo "Node list: $SLURM_NODELIST"
10 echo "CPUs/node: $SLURM_CPUS_ON_NODE"
11 echo "GPUs visible: $CUDA_VISIBLE_DEVICES"
12
13 echo "==== nvidia-smi ===="
14 nvidia-smi
15
16 echo "==== lscpu ===="
17 lscpu
18
19 echo "==== memory ===="
20 free -h

```

Aspect	CPU	Single GPU	DataParallel (DP)	DistributedDataParallel (DDP)
Hardware	CPU cores only	1 GPU	Multiple GPUs (1 node)	Multiple GPUs (1+ nodes)
Processes	1	1	1	1 per GPU
GPU usage	None	Full single GPU	Automatic batch split	Explicit per-rank GPU
Multi-node support	No	No	No	Yes
Performance	Slowest	Fast	Medium	Fastest
Scalability	Poor	Limited	Limited	Excellent
Communication	None	None	CPU-based gather/scatter	NCCL all-reduce
Gradient sync	N/A	N/A	Implicit (inefficient)	Explicit (efficient)
Memory efficiency	High	Medium	Low (replication overhead)	High
Fault isolation	N/A	N/A	Poor	Good
Code complexity	Very low	Low	Low	Medium
Launch method	<code>python</code>	<code>python</code>	<code>python</code>	<code>torchrun</code> / <code>srun</code>
Best use case	Debugging, tests	Small models	Prototyping	Large-scale training
Recommended for HPC	No	Limited	No	Yes

Table 2.3: Comparison of CPU, single-GPU, DataParallel (DP), and DistributedDataParallel (DDP) execution modes in PyTorch.

Aspect	CPU	Single GPU	DataParallel (DP)	DistributedDataParallel (DDP)
Hardware	CPU cores only	1 GPU	Multiple GPUs (1 node)	Multiple GPUs (1+ nodes)
Processes	1	1	1	1 per GPU
GPU usage	None	Full single GPU	Automatic batch split	Explicit per-rank GPU
Multi-node support	No	No	No	Yes
Performance	Slowest	Fast	Medium	Fastest
Scalability	Poor	Limited	Limited	Excellent
Communication	None	None	Gather/scatter overhead	NCCL all-reduce
Gradient sync	N/A	N/A	Implicit (less efficient)	Explicit (efficient)
Memory efficiency	High	Medium	Lower (replication overhead)	Medium (replicated per GPU)
Fault isolation	N/A	N/A	Poor	Better
Code complexity	Very low	Low	Low	Medium
Launch method	<code>python</code>	<code>python</code>	<code>python</code>	<code>torchrun/srun</code>
Best use case	Debugging, tests	Small models	Prototyping	Large-scale training
Recommended for HPC	No	Limited	No	Yes

Table 2.4: Comparison of CPU, single-GPU, DataParallel (DP), and DistributedDataParallel (DDP) execution modes in PyTorch.

2.8 PyTorch DataParallel (DP) on SLURM

This section describes PyTorch **DataParallel (DP)**, a simpler multi-GPU approach compared to Distributed Data Parallel (DDP). DP uses a *single Python process* and automatically splits each batch across multiple GPUs on the same node.

Important: DP is suitable for quick experiments on a *single node* only. For serious training, large models, or multi-node jobs, prefer DDP (Section 2.9).

2.8.1 When to Use DataParallel

- Single node with multiple GPUs.
- Prototyping or debugging multi-GPU behavior.
- Minimal code changes from single-GPU training.

2.8.2 When *Not* to Use DataParallel

- Multi-node training (DP does not scale across nodes).
- Large models or long training runs (DDP is faster and more stable).
- When precise performance benchmarking matters.

2.8.3 Minimal DataParallel Training Skeleton

Save as `train_dp.py`. This example assumes a single node with multiple GPUs.

Listing 2.1: Minimal PyTorch DataParallel training skeleton (`train_dp.py`)

```

1 import torch
2 import torch.nn as nn
3 from torch.utils.data import DataLoader
4
5 def build_model():
6     # Replace with your FCN / VAE model
7     return nn.Identity()
8
9 def build_dataset(split="train"):
10    # Replace with your dataset
11    raise NotImplementedError
12
13 def main():
14    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
15
16    model = build_model()
17
18    if torch.cuda.device_count() > 1:
19        print(f"Using DataParallel on {torch.cuda.device_count()} GPUs")
20        model = nn.DataParallel(model)
21
22    model = model.to(device)
23
24    train_ds = build_dataset("train")
25    train_loader = DataLoader(
26        train_ds,
27        batch_size=16,
28        shuffle=True,
29        num_workers=4,
30        pin_memory=True,
31        drop_last=True,
32    )
33
34    optim = torch.optim.Adam(model.parameters(), lr=1e-3)
35
36    model.train()
37    for epoch in range(5):
38        for batch in train_loader:
39            X, Y = batch
40            X = X.to(device, non_blocking=True)
41            Y = Y.to(device, non_blocking=True)
42
43            optim.zero_grad(set_to_none=True)
44            Yhat = model(X)
45            loss = torch.mean((Yhat - Y) ** 2)
46            loss.backward()
47            optim.step()
48
49            print(f"Epoch {epoch} done. loss={loss.item():.6f}")
50
51 if __name__ == "__main__":
52     main()

```

2.8.4 SLURM Script for DataParallel (single node)

DP requires **one Python process only**. Do *not* use `torchrun` or multiple tasks.

Listing 2.2: SLURM script for DataParallel (`dp_1node.slurm`)

```

1  #!/bin/bash
2  #SBATCH --job-name=dp_1node
3  #SBATCH --output=dp_1node.out.%j
4  #SBATCH --error=dp_1node.err.%j
5  #SBATCH -N 1
6  #SBATCH -p kimg
7  #SBATCH --gres=gpu:4
8  #SBATCH --cpus-per-task=8
9  #SBATCH --time=04:00:00
10
11  echo "Host: \$(hostname)"
12  echo "CUDA_VISIBLE_DEVICES: \$(CUDA_VISIBLE_DEVICES)"
13
14  source ~/myEnvs/tf_env/bin/activate
15
16  export OMP_NUM_THREADS=\$SLURM_CPUS_PER_TASK
17
18  # Single process only
19  python train_dp.py
20
21  deactivate

```

2.8.5 DP vs DDP: Practical Comparison

Feature	DataParallel (DP)	DistributedDataParallel (DDP)
Processes	1 total	1 per GPU
Multi-node support	No	Yes
Performance	Slower	Faster
Scalability	Limited	Excellent
Failure isolation	Poor	Good
Recommended for	Prototyping	Production / research

Table 2.5: Comparison between DataParallel and DistributedDataParallel.

2.8.6 Common DP Pitfalls

- Do not combine DP with `torchrun` or `srun -n > 1`.
- Expect slower scaling beyond 2–4 GPUs.
- Avoid DP for multi-node jobs.
- Always move the model to GPU *after* wrapping with `DataParallel`.

2.8.7 Rule of Thumb

- **1 node, quick test** → `DataParallel`
- **Anything serious or multi-node** → `DDP`

2.9 PyTorch Distributed Data Parallel (DDP) on SLURM

This section shows a minimal, practical recipe for PyTorch DDP on an HPC cluster. DDP runs one Python process per GPU and synchronizes gradients efficiently.

2.9.1 What You Need (Checklist)

- Request **multiple GPUs** in SLURM (e.g., `-gres=gpu:4`).
- Launch with `torchrun` (recommended) or `srun` + environment variables.
- In Python: call `init_process_group`, set the per-rank GPU, wrap model with `DistributedDataParallel`.
- Use `DistributedSampler` for training (and validation if you want correct global metrics quickly).

2.9.2 Minimal DDP Training Skeleton (single file)

Save as `train_ddp.py`. Replace `build_model()`, `build_dataset()`, and the loss/optimizer with your own code.

Listing 2.3: Minimal PyTorch DDP training skeleton (train_ddp.py)

```

1  import os
2  import torch
3  import torch.nn as nn
4  import torch.distributed as dist
5  from torch.nn.parallel import DistributedDataParallel as DDP
6  from torch.utils.data import DataLoader
7  from torch.utils.data.distributed import DistributedSampler
8
9  # -----
10 # 1) DDP setup / teardown
11 # -----
12 def ddp_setup():
13     """
14     Works with: torchrun (sets RANK, LOCAL_RANK, WORLD_SIZE, MASTER_ADDR, MASTER_PORT
15     ).
16     """
17     if dist.is_available() and not dist.is_initialized():
18         dist.init_process_group(backend="nccl") # GPUs -> nccl
19
20     rank = int(os.environ.get("RANK", "0"))
21     local_rank = int(os.environ.get("LOCAL_RANK", "0"))
22     world_size = int(os.environ.get("WORLD_SIZE", "1"))
23
24     if torch.cuda.is_available():
25         torch.cuda.set_device(local_rank)
26         device = torch.device(f"cuda:{local_rank}")
27     else:
28         device = torch.device("cpu")
29
30     return device, rank, local_rank, world_size
31
32 def ddp_cleanup():
33     if dist.is_available() and dist.is_initialized():
34         dist.destroy_process_group()
35
36 # -----
37 # 2) Your builders (replace)
38 # -----
39 def build_model():
40     # Replace with your FCN/VAE model builder
41     return nn.Identity()
42
43 def build_dataset(split="train"):
44     # Replace with your dataset (e.g., CleanImageFolder / PairedImageFolder)
45     # Must return a torch.utils.data.Dataset
46     raise NotImplementedError
47
48 # -----
49 # 3) Main training loop
50 # -----
51 def main():
52     device, rank, local_rank, world = ddp_setup()
53     is_ddp = (world > 1)
54
55     model = build_model().to(device)
56     if is_ddp:
57         model = DDP(model, device_ids=[local_rank], output_device=local_rank)

```

2.9.3 SLURM Script for DDP (single node, 4 GPUs)

This launches **one process per GPU** on the same node.

Listing 2.4: SLURM script for DDP on 1 node (`ddp_1node.slurm`)

```

1  #!/bin/bash
2  #SBATCH --job-name=ddp_1node
3  #SBATCH --output=ddp_1node.out.%j
4  #SBATCH --error=ddp_1node.err.%j
5  #SBATCH -N 1
6  #SBATCH -p kimg
7  #SBATCH --gres=gpu:4
8  #SBATCH --cpus-per-task=4
9  #SBATCH --time=04:00:00
10
11  echo "Host: \$(hostname)"
12  echo "Node list: \${SLURM_NODELIST}"
13  echo "CUDA_VISIBLE_DEVICES: \${CUDA_VISIBLE_DEVICES}"
14
15  source ~/myEnvs/tf_env/bin/activate
16
17  # One process per GPU:
18  export OMP_NUM_THREADS=\${SLURM_CPUS_PER_TASK}
19
20  # torchrun is recommended.
21  # --standalone works for single-node. For multi-node, use rendezvous settings shown
    below.
22  srun torchrun \
23    --standalone \
24    --nproc_per_node=4 \
25    train_ddp.py
26
27  deactivate

```

2.9.4 SLURM Script for DDP (2 nodes, 4 GPUs per node)

For multi-node DDP, you need a rendezvous endpoint (master address/port). A common pattern is to use the first node in `\${SLURM_NODELIST}` as the master.

Listing 2.5: SLURM script for DDP on 2 nodes (ddp_2node.slurm)

```

1  #!/bin/bash
2  #SBATCH --job-name=ddp_2node
3  #SBATCH --output=ddp_2node.out.%j
4  #SBATCH --error=ddp_2node.err.%j
5  #SBATCH -N 2
6  #SBATCH -p kimg
7  #SBATCH --gres=gpu:4
8  #SBATCH --cpus-per-task=4
9  #SBATCH --time=06:00:00
10
11  source ~/myEnvs/tf_env/bin/activate
12
13  # Pick master node and port
14  MASTER_ADDR=$(scontrol show hostnames "$SLURM_NODELIST" | head -n 1)
15  MASTER_PORT=29500
16
17  echo "MASTER_ADDR=$MASTER_ADDR"
18  echo "MASTER_PORT=$MASTER_PORT"
19
20  export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
21
22  # Total processes = nodes * gpus_per_node = 2 * 4 = 8
23  srun torchrun \
24    --nnodes=2 \
25    --nproc_per_node=4 \
26    --rdzv_backend=c10d \
27    --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT \
28    train_ddp.py
29
30  deactivate

```

2.9.5 Common DDP Pitfalls (Fast Debug List)

- **Do not use DataParallel with DDP.** Use one or the other.
- Call `DistributedSampler(...)` for training and call `set_epoch(epoch)` each epoch.
- Set the per-process GPU via `LOCAL_RANK` and `torch.cuda.set_device(local_rank)`.
- **Print/checkpoint only on rank 0** to avoid duplicates and file corruption.
- If validation metrics must be global, either:
 - use `DistributedSampler` for validation and `all_reduce` metric sums, or
 - run validation only on rank 0 (simpler but slower).

Chapter 3

Example Commands

3.1 Example Commands for File Transfer

This chapter provides practical, copy-pasteable examples for transferring files and directories between your local machine and the CRADLE cluster. These commands are typically run from a local terminal, not inside a SLURM job.

Useful HPC Commands for SLURM, Modules, Conda, and Remote Workflows

The following commands are commonly used when working on an HPC cluster. They help users inspect cluster resources, manage software modules, configure Conda environments, transfer files, and monitor SLURM jobs.

For additional examples of secure file transfer using `scp`, see the following reference:

<https://www.geeksforgeeks.org/linux-unix/scp-command-in-linux-with-examples/>

```
1
2 #####
3 # Inspect Cluster Resources
4 #####
5
6 # Show available partitions (queues), nodes, and their status
7 sinfo
8
9 # Example output fields:
10 # PARTITION -> queue name
11 # AVAIL -> whether it is available
12 # NODES -> number of nodes
13 # STATE -> idle, mix, alloc, drain
14
15
16 #####
17 # Environment Modules Management
18 #####
19
20 # Show all available software modules
21 module avail
22
23 # Show modules currently loaded in your environment
24 module list
```

```

25
26 # Load a software module (example: CUDA toolkit)
27 module load cuda/12.3
28
29 # Unload a specific module
30 module unload cuda/12.3
31
32 # Remove all loaded modules and reset the module environment
33 module purge
34
35
36 #####
37 # Creating Project Directories
38 #####
39
40 # Create a main project directory
41 mkdir -p ~/project_workspace
42
43 # Create a directory for storing datasets
44 ## -p means create parent directories as needed
45 mkdir -p ~/project_workspace/Data_Source
46
47
48 #####
49 # Installing Conda (Optional)
50 #####
51
52 # Download the full Anaconda distribution
53 wget https://repo.anaconda.com/archive/Anaconda3-2025.12-2-Linux-x86_64.sh
54
55 # OR download the lightweight Miniconda installer
56 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
57
58
59 #####
60 # Conda Environment Setup
61 #####
62
63 # Enable conda in the current shell session
64 # (required if conda is installed but not automatically initialized)
65 eval "$$(conda shell.bash hook)"
66
67 # Verify conda installation
68 conda --version
69
70 # Check which Python interpreter is currently active
71 # Useful for confirming the active Conda environment
72 which python
73 which python3
74
75 # Make conda initialization permanent for future logins
76 conda init bash
77
78

```

```

79 #####
80 # Disable Automatic Base Environment
81 #####
82
83 # Prevent the base environment from activating automatically
84 conda config --set auto_activate_base false
85
86 # Reload shell configuration
87 source ~/.bashrc
88
89
90
91
92
93
94
95 #####
96 # Running Jupyter Notebook on HPC
97 #####
98
99 # Start a Jupyter notebook on a remote machine
100 jupyter notebook --no-browser --port=8888
101
102
103 #####
104 # Create SSH Tunnel from Local Machine
105 #####
106
107 # Forward local port 8888 to the remote HPC node
108 ssh -L 8888:localhost:8888 username@login.cluster.edu
109
110 # If Jupyter is running on a compute node, first identify that node name
111 hostname
112
113 # Then create an SSH tunnel to that specific node through the login node
114 ssh -L 8888:localhost:8888 -J username@login.cluster.edu username@compute-node
115
116 # Open the URL printed by Jupyter in your local browser
117 http://localhost:8888/tree?token=YOUR_TOKEN_HERE
118
119
120
121 #####
122 # Copy Files and Folders to HPC with scp
123 #####
124
125 # Step 1: Log in to the HPC cluster and create destination directories
126 ssh yourUserName@login.cradle.university.edu
127
128 mkdir -p ~/project_workspace
129 mkdir -p ~/project_workspace/envs
130
131 # Step 2: From your local computer, copy files to the cluster
132 # Quotes are used because Windows paths often contain spaces

```

```

133
134 # Copy an entire local directory to the HPC cluster
135 # The -r option means "recursive", so the full folder and its contents are copied
136 scp -r "C:\Users\...\Data_Source" \
137     yourUserName@login.cradle.university.edu:/home/yourUserName/project_workspace
138
139 # Copy a single local text file
140 scp "C:\Users\...\mask_rcnn\install_steps.txt" \
141     yourUserName@login.cradle.university.edu:/home/yourUserName/project_workspace/
142     envs
143
144 # Copy another single local file
145 scp "C:\Users\...\Projects\HPC\install_hpc.txt" \
146     yourUserName@login.cradle.university.edu:/home/yourUserName/project_workspace/
147     envs
148
149 #####
150 # Fix Windows Line Ending Issues
151 #####
152
153 # Convert Windows CRLF line endings to Linux format
154 dos2unix job_script.sh
155
156 # Alternative method if dos2unix is unavailable
157 sed -i 's/\r$//' job_script.sh
158
159 #####
160 # zip / unzip checkpoints
161 #####
162
163 # Create zip file, excluding all .pth files
164 zip -r checkpoints.zip checkpoints/ -x "*.pth"
165
166 # Check contents before extracting
167 unzip -l checkpoints.zip
168
169 # Extract zip file
170 unzip checkpoints.zip
171
172 #####
173 # SLURM Job Monitoring
174 #####
175
176 # Check whether a specific job is still in the queue
177 squeue -j JOB_ID
178
179 # Show accounting information for a finished job
180 # (may be disabled on some clusters)
181 sacct -j JOB_ID
182
183
184 #####

```



```

185 # Inspect Job Output Files
186 #####
187
188 # List files in the current directory
189 ls
190
191 # Display the job's standard output
192 cat job_output.out
193
194 # Display the job's error output
195 cat job_error.err
196
197 # View large output files interactively
198 less job_output.out

```

These commands form a typical workflow when interacting with an HPC cluster: checking available resources, managing software modules and environments, running interactive applications such as Jupyter, and monitoring SLURM jobs.

3.1.1 Copy files from/to CRADLE

Copying a Single File from Local Machine to CRADLE

```

1 scp my_script.py \
2   yourUserName@login.cradle.university.edu:/home/yourUserName/projects/

```

This copies `my_script.py` from your current local directory into `/home/yourUserName/projects/` on the cluster.

Copying a Single File from CRADLE to Local Machine

```

1 scp yourUserName@login.cradle.university.edu:/home/yourUserName/projects/results.txt
2   .

```

The dot (.) represents your current local directory.

Copying an Entire Directory to CRADLE

To recursively copy a directory (for example, a project folder):

```

1 scp -r myProjectFolder/ \
2   yourUserName@login.cradle.university.edu:/home/yourUserName/projects/

```

This creates `/home/yourUserName/projects/myProjectFolder/` on the cluster.

Copying an Entire Directory from CRADLE to Local Machine

```

1 scp -r yourUserName@login.cradle.university.edu:/home/yourUserName/projects/
2   myProjectFolder/ \
3   ~/Desktop/

```

Copying an Entire Directory from CRADLE to the Current Local Folder

```

1  # Copy a full directory from CRADLE to the current folder on your local machine
2  scp -r yourUserName@login.cradle.university.edu:/home/yourUserName/projects/
   myProjectFolder/ .

```

Copying a Directory from CRADLE Without Replacing Existing Local Files

```

1  # Copy a directory from CRADLE to the current local folder
2  # Existing local files will not be overwritten
3  rsync -av --progress --ignore-existing yourUserName@login.cradle.university.edu:/home
   /yourUserName/projects/myProjectFolder/ .
4
5  # Copy only the contents inside myProjectFolder
6  rsync -av --ignore-existing user@server:/path/myProjectFolder/ .
7
8  # Copy the whole myProjectFolder directory
9  rsync -av --ignore-existing user@server:/path/myProjectFolder .
10
11
12 # Does not delete files that already exist in the destination
13 # but are missing from the source.
14 rsync -av source/ destination/
15
16
17 # To make the destination exactly match the source.
18 # WARNING: this deletes files in destination that are not in source.
19 rsync -av --delete checkpoints/ checkpoints_pc/
20
21
22 # Read size of a folder.
23 du -sh checkpoints
24
25
26 # Read size of each subfolder inside a folder.
27 du -h --max-depth=1 checkpoints
28
29
30 # Sort folder/subfolder sizes.
31 du -h --max-depth=1 checkpoints | sort -h

```

Copying a Directory from CRADLE Without Replacing Existing Local Files

```

1  # rsync options:
2  # -a : archive mode; copies recursively and preserves file metadata
3  # -v : verbose; shows what is being transferred
4  # -h : human-readable file sizes
5  # --progress : show transfer progress
6  # --ignore-existing : skip files that already exist in the destination
7
8  rsync -avh --progress --ignore-existing yourUserName@login.cradle.university.edu:/
   home/yourUserName/projects/myProjectFolder/ .

```

Transferring Job Output After Completion

After a SLURM job finishes, you often want to retrieve output files such as logs or trained models.

```
1 scp yourUserName@login.cradle.university.edu:/home/yourUserName/slurm_outputs/myJob.out.123456 \  
2 .
```

Here, 123456 is the SLURM job ID.

Copying Files Using Wildcards

You can use wildcards to transfer multiple files at once:

```
1 scp yourUserName@login.cradle.university.edu:/home/yourUserName/slurm_outputs/*.out.* \  
2 .
```

This copies all SLURM output files in the directory.

Move Folders

```
1 # Move source folders into a destination folder  
2 mv source_folder1 source_folder2 destination_folder/
```

Recommended Workflow with SLURM Jobs

A typical workflow looks like this:

1. Copy scripts to CRADLE using `scp` or `git clone`.
2. Submit the job using `sbatch`.
3. Monitor the job with `squeue`.
4. Copy results back to your local machine using `scp`.

Important Note: File transfers should generally be performed from the *login node*, not from inside running compute jobs, unless explicitly required.

3.1.2 Installing Anaconda or Miniconda on CRADLE

Download the installer from the login node:

```
1 cd /home/yourUserName  
2  
3 # Full Anaconda distribution  
4 wget https://repo.anaconda.com/archive/Anaconda3-2025.12-2-Linux-x86_64.sh  
5  
6 # OR lightweight Miniconda installer  
7 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh  
8  
9 # Clear Screen  
10 clear
```

Run the installer:

```

1  # Example for Anaconda
2  bash Anaconda3-2025.12-2-Linux-x86_64.sh
3
4  # Example for Miniconda
5  bash Miniconda3-latest-Linux-x86_64.sh
6
7
8  # Retart Terminal or run cmd:
9  source ~/.bashrc

```

Activating Conda in the Current Session

If you chose not to initialize Conda automatically during installation, enable it manually in the current shell:

Listing 3.1: Sample Code

```

1
2  eval "\$(/home/yourUserName/anaconda3/bin/conda shell.bash hook)"
3  conda --version
4
5  # Deactivate conda environment
6  conda deactivate
7
8  # Show current directory
9  pwd
10
11 # Show available cluster resources
12 sinfo
13
14 # Show all queued/running jobs
15 squeue
16
17 # Show queued/running jobs for a specific user
18 squeue -u $USER

```

Making Conda Available for Future Logins

To make Conda automatically available in every login session:

```

1  /home/yourUserName/anaconda3/bin/conda init bash
2  source ~/.bashrc
3  conda --version

```

3.1.3 Example SLURM Job Script Using a Conda Environment

The following example shows how to run a GPU-enabled Python program on the CRADLE cluster using a Conda environment.

Listing 3.2: Sample Code

```

[
# Don't forget to have the folders first ****
# mkdir -p logs ]

```

```

#!/bin/bash
#SBATCH --job-name=maskrcnn
#SBATCH --output=logs/maskrcnn.out.%j
#SBATCH --error=logs/maskrcnn.err.%j
#SBATCH -N 1
#SBATCH -p gpua30q
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=2
#SBATCH --time=02:00:00

echo "Host: $(hostname)"
echo "CUDA_VISIBLE_DEVICES: $CUDA_VISIBLE_DEVICES"

# initialize conda
# source ~/anaconda3/etc/profile.d/conda.sh
# OR
source \$(conda info --base)/etc/profile.d/conda.sh

# activate environment
conda activate mask_rcnn

echo "Python path: \$(which python)"

python train_maskrcnn.py
\end{codeblock}

\subsubsection{Select GPU}
\begin{codeblock}[numbers=none,xleftmargin=0pt,framexleftmargin=0pt]
# Select GPU (Optional) *****
#SBATCH --nodelist=gpu007

```

gpuinfo.sh

```
# save as gpuinfo.sh
#!/bin/bash
#SBATCH --job-name=gpuinfo
#SBATCH --output=gpuinfo.out.%j
#SBATCH --error=gpuinfo.err.%j
#SBATCH -p gpuq
#SBATCH --nodelist=gpu007
#SBATCH --gres=gpu:1
#SBATCH -c 1
#SBATCH --mem=2G
#SBATCH -t 00:05:00

echo "Date: \$(date)"
echo "Host: \$(hostname)"
echo "CUDA_VISIBLE_DEVICES=\$CUDA_VISIBLE_DEVICES"

nvidia-smi
nvidia-smi --query-gpu=index,name,memory.total,memory.free,memory.used,utilization.gpu --
format=csv
```

This script requests one GPU from the `gpua30q` partition and activates the `mask_rcnn` Conda environment before running the training script.

3.1.4 Sample Run code for sh

Listing 3.3: Sample Code

```
# Check general disk quota
quota -s
quota

# Check size of current directory
du -sh .

# Check size of your work directory
du -sh $WORK

# Check size of your scratch directory
du -sh $SCRATCH

# Check size of your home directory
du -sh $HOME

# Go to the working folder
cd folder

# List files in the current folder
ls
```

```
# List files in the current folder, organized with directories first
ls -lh --group-directories-first

# Remove old log directory
rm -rf logs

# Move a folder into another folder
mv folder to_folder/

# Move a file into another folder
mv file to_folder/

# Rename a folder
mv folder_old folder_new

# Rename a file
mv file_old file_new

# Create a fresh log directory
mkdir -p logs

# Open the scheduler script for editing
nano scheduler.sh

# Convert Windows line endings to Unix line endings
dos2unix scheduler.sh

# Submit the job to Slurm
sbatch scheduler.sh

# Check running or queued jobs for a specific user
squeue -u $USER

# Check job status
sacct -j job_id --format=JobID,JobName,Partition,State,ExitCode,Elapsed,Start,End
```

Listing 3.4: Sample Code

```
--partition=gpuq
or
-p gpuq

--odelist=gpu001
or
-w gpu001

--gres=gpu:1 request 1 GPU
-N 1 number of nodes
-c 4 CPUs per task
--cpus-per-task=4 same as above
--mem=16G memory
--time=01:00:00 wall time
--job-name=name job name
```

```

--output=file standard output file
--error=file error file

sbatch -p gpuq --nodelist=gpu003 --gres=gpu:1 -c 4 --mem=16G --time=02:00:00 myjob.sh

# Equivalent short form
sbatch -p gpuq -w gpu003 --gres=gpu:1 -c 4 --mem=16G --time=02:00:00 myjob.sh

# Full Name
squeue -u $USER -o "%.18i %.12P %.35j %.12u %.2t %.10M %.6D %R"

squeue -p gpul40q -o "%.18i %.9P %.20j %.8u %.2t %.10M %.6D %R %.20b"

# Time remaining
scontrol show job 45987 | grep -E "RunTime|TimeLimit|StartTime|EndTime"

for job in 46001 45999 45998 45997 45995 46000; do
    echo "===== "
    echo "JobID: $job"
    scontrol show job "$job" | grep -E "JobId=|JobName=|Partition=|RunTime=|TimeLimit
    =|StartTime=|EndTime=|NodeList="
done

#####
# Synchronize checkpoints folder between two Linux machines
#####

# -----
# 1. Copy remote checkpoints folder to current local directory
# Only copy missing files.
# Do not overwrite existing local files.
# Exclude .pth files and files larger than 20 MB.
# -----
rsync -avh --progress --ignore-existing \
    --exclude='*.pth' \
    --max-size=20M \
    user_name@ls6.tacc.utexas.edu:/work/user_number/user_name/ls6/shahriar/projects/
    project_name/checkpoints \
    .

# -----
# 2. Copy remote checkpoints folder to a specific local destination
# Only copy missing files.
# Do not overwrite existing local files.
# -----
rsync -avh --progress --ignore-existing \
    --exclude='*.pth' \
    --max-size=20M \
    user_name@ls6.tacc.utexas.edu:/work/user_number/user_name/ls6/shahriar/projects/

```



```

    project_name/checkpoints \
    destination/

# -----
# 3. Synchronize/update destination from remote
# Existing files may be updated if the remote version is newer/different.
# This does NOT delete extra files in destination.
# -----
rsync -avh --progress \
    --exclude='*.pth' \
    --max-size=20M \
    user_name@ls6.tacc.utexas.edu:/work/user_number/user_name/ls6/shahriar/projects/
    project_name/checkpoints \
    destination/

# Show drained/down nodes and their reasons
sinfo -R

# Show node state, GPU resources, and drain reason
sinfo -N -p gpuq,sxm,q,kin,q -o "%P %N %T %G %E"

# =====
# Check GPU details by providing partition and node name
# =====
# See job details
squeue -j job_id
squeue -j 50113
scontrol show job 50113
# For one finished job:
sacct -j 50113

# See Node info ===
sinfo -N -o "%N %P %t %C %m %G"
scontrol show node gpu011
scontrol show node c305-002 | grep -E "Gres|CfgTRES|AllocTRES|Partitions"

# Check who is using the node
squeue -w gpu011

# Start an interactive session on a specific GPU node
srun -p gpua30q -w gpu010 --gres=gpu:3 -t 00:30:00 --pty bash

srun -p gpul40q -w gpu012 --gres=gpu:1 -c 4 --mem=16G -t 00:30:00 --pty bash

# After entering the compute node, check the node name
hostname

# Show full GPU status
nvidia-smi

# Show compact GPU usage, VRAM, temperature, and power
nvidia-smi --query-gpu=index,name,utilization.gpu,memory.used,memory.free,memory.

```

```
total,temperature.gpu,power.draw,power.limit --format=csv
```

Using `tmux` for Long File Transfers

When copying large files or folders between systems, the transfer may stop if the terminal is closed. To avoid this, use `tmux`, which keeps the command running in a separate terminal session.

```
1 tmux new -s transfer
```

The command above creates a new `tmux` session named `transfer`. Inside this session, run the file transfer command, for example:

```
1 scp -r -o MACs=hmach-sha2-256 Data_Source username@remote.server.edu:/home/username/  
   project_folder
```

To detach from the session without stopping the transfer, press:

```
1 Ctrl + b  
2 d
```

The transfer will continue running in the background. To reconnect to the same session later, use:

```
1 tmux attach -t transfer
```

To list active `tmux` sessions, use:

```
1 tmux ls
```

After the transfer finishes, close the session by typing:

```
1 exit
```

Using `rsync` with `tmux` for Safe Dataset Transfer

For large dataset transfers, `rsync` is preferred over `scp` because it can skip files that already exist at the destination. Using `tmux` is also helpful because the transfer can continue running even if the terminal disconnects.

```

# Start a tmux session for the transfer
tmux new -s transfer

# Create the destination folder on the remote HPC
ssh -o MACs=hmac-sha2-256 username@ls6.tacc.utexas.edu \
"mkdir -p /work/projectID/username/ls6/project_folder/Data_Source/AMR_DATASET"

# Dry run first:
# This only shows which files would be copied.
# It does not actually copy anything because of the -n option.
rsync -avhn --progress --ignore-existing \
-e "ssh -o MACs=hmac-sha2-256" \
AMR_DATASET/ \
username@ls6.tacc.utexas.edu:/work/projectID/username/ls6/project_folder/Data_Source/
AMR_DATASET/

# Actual transfer:
# Same command as above, but without -n.
# This copies missing files and skips files that already exist.
rsync -avh --progress --ignore-existing \
-e "ssh -o MACs=hmac-sha2-256" \
AMR_DATASET/ \
username@ls6.tacc.utexas.edu:/work/projectID/username/ls6/project_folder/Data_Source/
AMR_DATASET/

# Detach from tmux without stopping the transfer:
# Press Ctrl+b, then press d

# Exclude both .pth files and files larger than 20 MB ***
rsync -av --exclude='*.pth' --max-size=20M source_folder/ username@remote:/path/to/
destination/

# Reconnect later to the same tmux session
tmux attach -t transfer

# After the transfer is finished, close the tmux session
exit

```

Here, `-n` means dry run, so the first `rsync` command only previews the transfer. The second command removes `-n`, so it performs the actual copy. The option `-ignore-existing` prevents existing destination files from being replaced.

3.1.5 Launching Jupyter Notebook on CRADLE

Start Jupyter Notebook on the remote machine (login or compute node):

```
jupyter notebook --no-browser --port=8888

or
# For GPU Node
jupyter notebook --no-browser --ip=0.0.0.0 --port=8890
```

The output will display a URL containing an authentication token, for example:

```
http://localhost:8888/tree?token=2c23d0
or
http://127.0.0.1:8888/tree?token=2c23d0
```

Creating an SSH Tunnel from Your Local Machine

On your local computer (Windows, macOS, or Linux), create a secure tunnel:

```
ssh -L 8888:localhost:8888 user@login.cradle.university.edu
or
# For GPU Node
ssh -L 8888:hostname:8888 user@login.cradle.university.edu

ssh -L 8888:gpu001:8888 user@login.cradle.university.edu
```

This forwards local port 8888 on your computer to port 8888 on the CRADLE server.

Accessing Jupyter in Your Web Browser

After the SSH tunnel is established, open your browser and navigate to:

```
1 http://localhost:8888/tree?token=2c23d0
2 or
3 # For GPU Node
4 http://127.0.0.1:8888/tree?token=2c23d0
```

Paste the token exactly as shown in the Jupyter output on CRADLE.

3.1.6 Troubleshooting Deleting a Folder

```
1 # Try deleting the folder
2 rm -rf logs
3
4 # If permission is denied, give yourself read, write, and execute permission
5 chmod -R u+rwX logs
6
7 # Delete the folder again
8 rm -rf logs
```

```
1 # Check write permission for the current folder
2 ls -ld .
3
4 # Check permission for the parent folder
5 ls -ld ..
6
7 # Add write permission for the current user
8 chmod u+w .
```

3.2 Checking Previous Slurm Job Submissions

Since Slurm accounting is disabled on Cradle, the `sacct` command cannot display completed-job history. However, previous job submissions can still be reviewed from the shell command history.

To display recent `sbatch` commands from the current terminal history, use:

```
1 history | grep sbatch
```

To search the saved Bash history file, use:

```
1 grep -h "sbatch" ~/.bash_history 2>/dev/null
```

These commands show previously executed `sbatch` submissions and can help identify the job scripts that were used. However, they do not provide complete information about job duration, GPU usage, memory consumption, or completion status.

3.2.1 Single Node, CPU-Only Job

```
1 #!/bin/bash
2 #SBATCH --job-name=cpu_only_job
3 #SBATCH --output=cpu_only_job.out.%j
4 #SBATCH --error=cpu_only_job.err.%j
5 #SBATCH -N 1
6 #SBATCH -p compute # CPU-only partition
7
8 #SBATCH --mail-user=username@gmail.com
9 #SBATCH --mail-type=BEGIN,END,FAIL
10
11 #SBATCH --time=01:00:00 # 1 hour
12
13 echo "Host: $(hostname)"
14 echo "Using CPU-only environment"
15 source ~/myEnvs/cpu_env/bin/activate
16
17 echo "Running CPU-only Python script"
18 python ~/projects/my_cpu_script.py
19
20 echo "Deactivating environment"
21 deactivate
```

Checking and Managing Modules on HPC

HPC systems often use environment modules to load software such as CUDA, Python, compilers, and MPI libraries. The following commands are useful for checking available modules, viewing loaded modules, and loading or unloading specific software.

```
# Show all available modules
module avail

# Show available CUDA modules only
module avail cuda

# Show currently loaded modules
module list

# Load a specific module
module load cuda/12.8

# Check loaded modules again
module list

# Unload a specific module
module unload cuda

# Remove all loaded modules
module purge

# Reload the default TACC environment if needed
module load TACC

# Save the currently loaded collection of modules
module save


echo \${HOME} # Without backslash
echo \${WORK} # Without backslash
echo \${SCRATCH} # Without backslash


cdh # Goto home directory
cdw # Goto work directory
cds # Goto scratch directory


conda --version
#conda 25.11.1
conda info --base
#/work/11394/user/wd/shahriar/anaconda3


# idle gpu
sinfo -t idle
```

Single Node, Single GPU Job

```
#!/bin/bash
#SBATCH --job-name=single_gpu_job
#SBATCH --output=single_gpu_job.out.%j
#SBATCH --error=single_gpu_job.err.%j
#SBATCH -N 1
#SBATCH -p kimg # GPU partition
#SBATCH --gres=gpu:1 # 1 GPU

#SBATCH --mail-user=username@gmail.com
#SBATCH --mail-type=BEGIN,END,FAIL

#SBATCH --time=02:00:00 # 2 hours

echo "Host: $(hostname)"
echo "CUDA_VISIBLE_DEVICES: $CUDA_VISIBLE_DEVICES"

echo "Activating GPU TensorFlow environment"
source ~/myEnvs/tf_env/bin/activate

echo "Running single-GPU training script"
python ~/projects/train_single_gpu.py

echo "Deactivating environment"
deactivate
```

Single Node, Multiple GPUs (e.g., 4 GPUs)

```
#!/bin/bash
#SBATCH --job-name=multi_gpu_job
#SBATCH --output=multi_gpu_job.out.%j
#SBATCH --error=multi_gpu_job.err.%j
#SBATCH -N 1
#SBATCH -p kimg # GPU partition
#SBATCH --gres=gpu:4 # 4 GPUs on the same node
#SBATCH --time=04:00:00 # 4 hours

echo "Host: $(hostname)"
echo "CUDA_VISIBLE_DEVICES: $CUDA_VISIBLE_DEVICES"

echo "Activating GPU TensorFlow environment"
source ~/myEnvs/tf_env/bin/activate

echo "Running multi-GPU training script (e.g., data parallel)"
python ~/projects/train_multi_gpu.py --num-gpus 4

echo "Deactivating environment"
deactivate
```

Multiple Nodes, CPU-Only (e.g., MPI or Distributed CPU Job)

```
1  #!/bin/bash
2  #SBATCH --job-name=cpu_multi_node_job
3  #SBATCH --output=cpu_multi_node_job.out.%j
4  #SBATCH --error=cpu_multi_node_job.err.%j
5  #SBATCH -N 2 # 2 nodes
6  #SBATCH -p compute # CPU-only partition
7  #SBATCH --time=03:00:00 # 3 hours
8
9  echo "SLURM_NODELIST: $SLURM_NODELIST"
10 echo "SLURM_NNODES: $SLURM_NNODES"
11
12 echo "Activating CPU-only environment"
13 source ~/myEnvs/cpu_env/bin/activate
14
15 # Example: launch an MPI or distributed job
16 # (replace with your actual launcher, e.g., srun mpirun, torchrun, etc.)
17 echo "Running distributed CPU job"
18 srun python ~/projects/distributed_cpu_job.py
19
20 echo "Deactivating environment"
21 deactivate
```

Multiple Nodes, Multiple GPUs (Distributed GPU Training)

```
1  #!/bin/bash
2  #SBATCH --job-name=multi_node_multi_gpu
3  #SBATCH --output=multi_node_multi_gpu.out.%j
4  #SBATCH --error=multi_node_multi_gpu.err.%j
5  #SBATCH -N 2 # 2 nodes
6  #SBATCH -p kimg # GPU partition
7  #SBATCH --gres=gpu:4 # 4 GPUs per node (total 8 GPUs)
8  #SBATCH --time=06:00:00 # 6 hours
9
10 echo "SLURM_NODELIST: $SLURM_NODELIST"
11 echo "SLURM_NNODES: $SLURM_NNODES"
12 echo "CUDA_VISIBLE_DEVICES: $CUDA_VISIBLE_DEVICES"
13
14 echo "Activating GPU TensorFlow environment"
15 source ~/myEnvs/tf_env/bin/activate
16
17 # Example using srun to launch a distributed training job
18 # Replace with your actual distributed launcher (e.g., torchrun, horovodrun)
19 echo "Running multi-node, multi-GPU training"
20 srun python ~/projects/train_distributed_gpu.py \
21     --nodes $SLURM_NNODES \
22     --gpus-per-node 4
23
24 echo "Deactivating environment"
25 deactivate
```


3.2.2 Quick Mapping: Which Example to Use?

- **Create new CPU environment:** Listing in Section [2.7.1](#).
- **Clone shared GPU TF environment:** Listing in Section [2.7.2](#).
- **CPU-only job:** Section [3.2.1](#).
- **Single-GPU job:** Section [3.2.1](#).
- **Single node, multiple GPUs:** Section [3.2.1](#).
- **Multi-node CPU job:** Section [3.2.1](#).
- **Multi-node, multi-GPU job:** Section [3.2.1](#).

3.3 Summary

In this tutorial, you have seen:

- How to write and submit a basic SLURM job script that runs a TensorFlow program.
- How to interpret key `#SBATCH` directives: number of nodes, partition, GPU requests, and time limits.
- Why shared environments on HPC clusters are read-only and how to safely extend them by cloning to your home directory.
- How to create your own Python virtual environments from scratch and how to connect them to SLURM job scripts.

This workflow lets you run fast, GPU-enabled jobs using cluster-optimized software while keeping your custom Python packages isolated and safe.

Chapter 4

Deploying PyTorch Models with ONNX, Pygame, and Standalone Executables

4.1 Introduction

Training and deployment are separate stages of a machine-learning workflow. A model may be trained with PyTorch on an HPC cluster or workstation and then used on another computer without installing PyTorch on that computer. The trained model cannot, however, execute without any supporting software. It must be converted to a portable representation and executed by an inference runtime.

For a Python-based desktop application, a practical deployment path is:

PyTorch training → ONNX export → ONNX Runtime inference → Pygame GUI → standalone executable.

In this workflow, PyTorch is used only on the development computer. The target computer runs the exported model through ONNX Runtime. A packaging tool such as PyInstaller can then bundle the application and its dependencies so that the user does not need to install Python, PyTorch, Pygame, or ONNX Runtime separately.

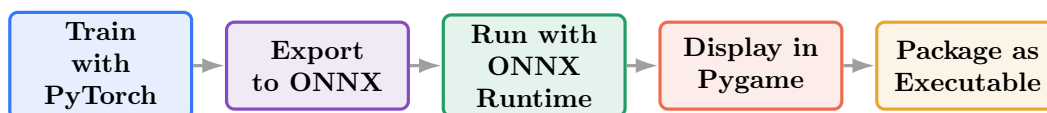


Figure 4.1: End-to-end workflow for deploying a PyTorch model without requiring PyTorch on the target computer.

4.2 Meaning of Deployment Without PyTorch

A saved PyTorch checkpoint such as `best_model.pth` generally contains trained parameters, but it does not provide an independent program that can perform inference. Loading that checkpoint normally requires the original model architecture and the PyTorch library. Exporting the model to ONNX stores a portable computational graph together with the trained parameters. ONNX Runtime can then execute that graph without importing PyTorch.

Component	Development or training computer	Target computer
Python	Required for the example workflow	Not separately required after packaging
PyTorch	Required for training and ONNX export	Not required
ONNX Runtime	Used to validate the exported model	Required internally, but it may be bundled
Pygame	Used to develop and test the interface	Required internally, but it may be bundled
PyInstaller	Required only when building the executable	Not required
Model checkpoint	Used to create the ONNX model	Not required after successful export
ONNX model	Created during export	Required, either beside or inside the executable

Table 4.1: Software and files needed during development and deployment.

4.2.1 Alternative Deployment Options

ONNX is not the only deployment option. The appropriate choice depends on the target hardware, programming language, model operators, and required performance.

Method	Target runtime	Main advantage	Important limitation
ONNX	ONNX Runtime	Portable across operating systems and programming languages	Some model operations may need modification during export
TorchScript or exported PyTorch program	LibTorch or a compatible PyTorch runtime	Retains a PyTorch-centered workflow	PyTorch-related runtime libraries are still needed
TensorRT	NVIDIA TensorRT	High inference performance on supported NVIDIA GPUs	Hardware and software compatibility requirements are stricter
OpenVINO	OpenVINO Runtime	Useful optimization options for Intel hardware	Conversion and operator support must be checked
Standalone Python package	Bundled Python and libraries	Straightforward for desktop prototypes	Executable size can be large

Table 4.2: Common choices for deploying a model trained in PyTorch.

For a first desktop application, CPU-based ONNX Runtime is usually the easiest option to distribute and troubleshoot.

4.3 Step-by-Step Deployment Plan

Table 4.3 separates the tasks performed on the training computer from those performed on the deployment computer.

Step	Computer	Action	Purpose
1	Training PC or HPC	Train and save the best model	Produces the learned parameters used for inference
2	Training PC	Recreate the model architecture	Provides the computation structure associated with the checkpoint
3	Training PC	Load the checkpoint and call <code>model.eval()</code>	Restores the trained parameters and enables inference behavior
4	Training PC	Create a representative sample input	Defines the expected input shape during export
5	Training PC	Export the model to <code>model.onnx</code>	Produces a portable model graph and weights
6	Training PC	Compare PyTorch and ONNX outputs	Detects conversion errors before deployment
7	Development PC	Build and test the Pygame application	Connects input, preprocessing, inference, postprocessing, and display
8	Development PC	Build a standalone executable	Bundles the program and its runtime dependencies
9	Target PC	Copy and run the executable	Performs inference without a separate PyTorch or Python installation
10	Target PC	Test representative inputs	Confirms correct behavior on the actual deployment hardware

Table 4.3: Recommended steps for exporting and deploying a PyTorch model.

4.4 Recommended Project Structure

A small project can be organized as follows:

Listing 4.1: Example project structure

```

1 pygame_onnx_app/
2 |-- app.py
3 |-- model_definition.py
4 |-- export_to_onnx.py
5 |-- model.onnx
6 |-- config.json
7 |-- requirements_development.txt
8 |-- requirements_runtime.txt
9 |-- build_windows.bat
10 +-- build_linux.sh

```

The files `model_definition.py` and `export_to_onnx.py` belong to the development side of the workflow. The deployed application uses `app.py`, `model.onnx`, and `config.json`. PyInstaller collects these files and the required libraries when it creates the executable.

4.5 Creating and Exporting the Model

4.5.1 Example Model Definition

Listing 4.2 defines a small four-class semantic segmentation model. It is included only to demonstrate the export and GUI workflow. A real project must use the same architecture that was used to create the trained checkpoint.

Listing 4.2: Small segmentation model for testing the deployment workflow

```

1 import torch
2 from torch import nn
3
4
5 class TinySegmentationModel(nn.Module):
6     def __init__(self, num_classes=4):
7         super().__init__()
8         self.network = nn.Sequential(
9             nn.Conv2d(3, 16, kernel_size=3, padding=1),
10            nn.ReLU(inplace=True),
11            nn.Conv2d(16, 32, kernel_size=3, padding=1),
12            nn.ReLU(inplace=True),
13            nn.Conv2d(32, num_classes, kernel_size=1),
14        )
15
16    def forward(self, images):
17        return self.network(images)
18
19
20 def build_model(num_classes=4):
21     return TinySegmentationModel(num_classes)

```

4.5.2 Loading the Checkpoint

The architecture must be constructed before its parameters are loaded. The checkpoint format may vary across training programs. Some scripts save a raw state dictionary, whereas others store it under a key such as `model_state_dict` or `state_dict`.

Listing 4.3: Loading a PyTorch checkpoint before export

```

1 import torch
2 from model_definition import build_model
3
4 model = build_model(num_classes=4)
5
6 checkpoint = torch.load(
7     "best_model.pth",
8     map_location="cpu",
9     weights_only=True,
10 )
11
12 if "model_state_dict" in checkpoint:
13     checkpoint = checkpoint["model_state_dict"]
14 elif "state_dict" in checkpoint:
15     checkpoint = checkpoint["state_dict"]
16
17 model.load_state_dict(checkpoint)
18 model.eval()

```

If the checkpoint is already a raw state dictionary, the conditional branches should be adjusted to match the checkpoint structure.

4.5.3 Exporting to ONNX

The sample tensor must match the number of channels and spatial dimensions expected by the model. Listing 4.4 uses one RGB image with a height of 192 pixels and a width of 512 pixels.

Listing 4.4: Exporting the trained model to ONNX

```
1 sample = torch.randn(1, 3, 192, 512)
2
3 torch.onnx.export(
4     model,
5     (sample,),
6     "model.onnx",
7     input_names=["image"],
8     output_names=["logits"],
9     opset_version=18,
10    dynamo=True,
11 )
```

The input and output names make the exported graph easier to inspect and use. The example uses a fixed input size. Dynamic shapes may be introduced when the application must accept multiple image sizes, but fixed shapes are simpler for an initial deployment.

4.5.4 Checking Numerical Agreement

Export should always be followed by a comparison of the outputs produced by PyTorch and ONNX Runtime for the same input.

Listing 4.5: Comparing PyTorch and ONNX outputs

```
1 import numpy as np
2 import onnxruntime as ort
3
4 with torch.inference_mode():
5     torch_output = model(sample).cpu().numpy()
6
7 session = ort.InferenceSession(
8     "model.onnx",
9     providers=["CPUExecutionProvider"],
10 )
11 onnx_output = session.run(
12     None,
13     {"image": sample.numpy()},
14 )[0]
15
16 maximum_error = np.max(
17     np.abs(torch_output - onnx_output)
18 )
19 print("Maximum absolute difference:", maximum_error)
```

A small floating-point difference is expected because runtimes may use different computational kernels. A large difference requires investigation. Possible causes include unsupported operators, a mismatched model mode, different preprocessing, or an incorrect output tensor.

4.6 Designing the Pygame Inference Application

The application performs six operations for every input image:

1. load the image;
2. convert it to RGB and resize it;
3. apply the same normalization used during training;
4. run the ONNX model;
5. convert logits to class labels and colors; and
6. display or save the result.

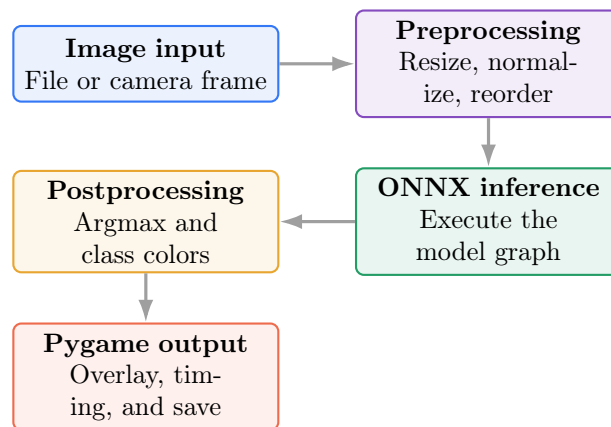


Figure 4.2: Data flow inside the Pygame semantic-segmentation application.

4.6.1 Configuration File

Preprocessing and class definitions should be stored outside the application code so they can be checked and updated easily.

Listing 4.6: Contents of config.json

```
1 {
2   "input_width": 512,
3   "input_height": 192,
4   "mean": [0.485, 0.456, 0.406],
5   "std": [0.229, 0.224, 0.225],
6   "class_names": [
7     "Road",
8     "Sidewalk",
9     "Cycle Path",
10    "Other"
11  ],
12  "colors": [
13    [128, 64, 128],
14    [244, 35, 232],
15    [70, 130, 180],
16    [107, 142, 35]
17  ]
18 }
```

The mean, standard deviation, class order, and input resolution must match training exactly. A change in class order can produce a visually plausible but incorrect mask.

4.6.2 Locating Bundled Files

When PyInstaller creates a one-file application, bundled resources are extracted to a temporary directory while the program is running. The helper in Listing 4.7 works both during development and inside the packaged application.

Listing 4.7: Locating model and configuration files

```
1 import sys
2 from pathlib import Path
3
4
5 def resource_path(name):
6     if getattr(sys, "frozen", False):
7         base = Path(sys._MEIPASS)
8     else:
9         base = Path(__file__).resolve().parent
10    return base / name
```

4.6.3 Loading ONNX Runtime

The deployed GUI does not import PyTorch. The model is loaded directly with an ONNX Runtime inference session.

Listing 4.8: Loading the ONNX model in the GUI

```

1 import json
2 import numpy as np
3 import onnxruntime as ort
4
5
6 class Segmenter:
7     def __init__(self):
8         with resource_path("config.json").open(
9             "r", encoding="utf-8"
10         ) as file:
11             self.config = json.load(file)
12
13             self.width = self.config["input_width"]
14             self.height = self.config["input_height"]
15             self.mean = np.asarray(
16                 self.config["mean"], dtype=np.float32
17             ).reshape(1, 1, 3)
18             self.std = np.asarray(
19                 self.config["std"], dtype=np.float32
20             ).reshape(1, 1, 3)
21             self.colors = np.asarray(
22                 self.config["colors"], dtype=np.uint8
23             )
24
25             self.session = ort.InferenceSession(
26                 str(resource_path("model.onnx")),
27                 providers=["CPUExecutionProvider"],
28             )
29             self.input_name = (
30                 self.session.get_inputs()[0].name
31             )

```

4.6.4 Preprocessing, Inference, and Postprocessing

Listing 4.9 shows the core prediction method. It belongs inside the `Segmenter` class and returns a colored mask, an overlay, and the measured model execution time.

Listing 4.9: Running semantic-segmentation inference

```
1 def predict(self, image):
2     original = image.convert("RGB")
3     resized = original.resize(
4         (self.width, self.height),
5         Image.Resampling.BILINEAR,
6     )
7
8     array = np.asarray(
9         resized, dtype=np.float32
10    ) / 255.0
11    array = (array - self.mean) / self.std
12    tensor = np.transpose(
13        array, (2, 0, 1)
14    )[None, ...].astype(np.float32)
15
16    started = time.perf_counter()
17    output = self.session.run(
18        None, {self.input_name: tensor}
19    )[0]
20    elapsed_ms = (
21        time.perf_counter() - started
22    ) * 1000.0
23
24    mask = np.argmax(output[0], axis=0)
25    color_mask = Image.fromarray(
26        self.colors[mask].astype(np.uint8)
27    )
28    color_mask = color_mask.resize(
29        original.size,
30        Image.Resampling.NEAREST,
31    )
32    overlay = Image.blend(
33        original, color_mask, alpha=0.45
34    )
35    return color_mask, overlay, elapsed_ms
```

This method requires `import time` and `from PIL import Image`. It assumes that the ONNX output has the shape `[batch, classes, height, width]`. Models that return a dictionary, multiple tensors, or an already-decoded label map require a corresponding adjustment.

4.6.5 Displaying a PIL Image in Pygame

Listing 4.10: Converting an image for Pygame display

```

1 import pygame
2
3
4 def pil_to_surface(image, panel_size):
5     fitted = image.copy()
6     fitted.thumbnail(
7         panel_size,
8         Image.Resampling.LANCZOS,
9     )
10    surface = pygame.image.frombuffer(
11        fitted.tobytes(),
12        fitted.size,
13        "RGB",
14    ).copy()
15
16    canvas = pygame.Surface(panel_size)
17    canvas.fill((42, 47, 56))
18    canvas.blit(
19        surface,
20        surface.get_rect(
21            center=canvas.get_rect().center
22        ),
23    )
24    return canvas

```

4.6.6 Keeping the Interface Responsive

Pygame processes window events in its main loop. If inference is performed inside that loop, the window may stop responding until the prediction finishes. A worker thread can perform inference while the main thread continues to draw the interface.

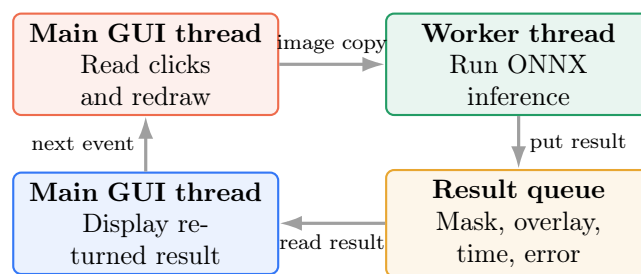


Figure 4.3: Threaded inference keeps the Pygame event loop responsive.

Listing 4.11: Running inference in a worker thread

```
1 import queue
2 import threading
3
4 result_queue = queue.Queue()
5 processing = False
6
7
8 def inference_worker(segmenter, image):
9     try:
10         mask, overlay, elapsed = (
11             segmenter.predict(image)
12         )
13         result_queue.put(
14             (mask, overlay, elapsed, None)
15         )
16     except Exception as error:
17         result_queue.put(
18             (None, None, None, str(error))
19         )
20
21
22 processing = True
23 threading.Thread(
24     target=inference_worker,
25     args=(segmenter, original.copy()),
26     daemon=True,
27 ).start()
```

The main loop can check the queue without blocking:

Listing 4.12: Collecting the completed inference result

```
1 try:
2     mask, overlay, elapsed, error = (
3         result_queue.get_nowait()
4     )
5     processing = False
6
7     if error:
8         status = "Inference failed: " + error
9     else:
10         result_surface = pil_to_surface(
11             overlay, panel_size
12         )
13         status = (
14             f"Inference completed in "
15             f"{elapsed:.1f} ms"
16         )
17 except queue.Empty:
18     pass
```

4.7 Installing the Development Dependencies

Create a virtual environment on the development computer:

Listing 4.13: Creating the development environment

```
1 python -m venv .venv
```

Activate it on Windows:

Listing 4.14: Activating the environment on Windows

```
1 .venv\Scripts\activate
```

Activate it on Linux:

Listing 4.15: Activating the environment on Linux

```
1 source .venv/bin/activate
```

Install the development packages:

Listing 4.16: Installing export, runtime, GUI, and packaging packages

```
1 python -m pip install \  
2     torch \  
3     onnx \  
4     onnxscript \  
5     onnxruntime \  
6     numpy \  
7     pillow \  
8     pygame \  
9     pyinstaller
```

After `model.onnx` has been created, test the application before packaging:

Listing 4.17: Testing the Pygame application

```
1 python app.py
```

4.8 Building a Standalone Runtime

The term *standalone* means that the required interpreter and libraries are distributed with the application. It does not mean that the executable contains no runtime components. PyInstaller analyzes the imports in `app.py`, collects the required Python modules and native libraries, and creates a distributable application.

4.8.1 Windows Build

Run the following command from the project directory:

Standalone application

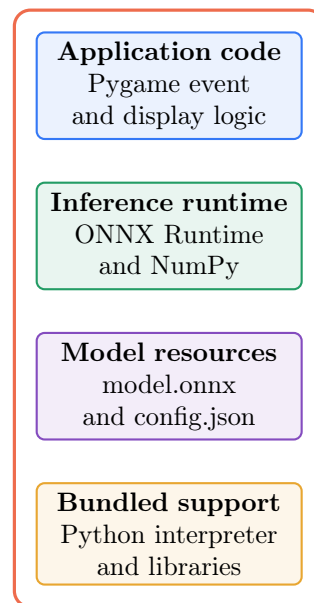


Figure 4.4: Main components bundled into the standalone application.

Listing 4.18: Building a standalone Windows application

```

1 python -m PyInstaller \
2   --noconfirm \
3   --clean \
4   --onefile \
5   --windowed \
6   --name SegmentationApp \
7   --collect-all onnxruntime \
8   --add-data "model.onnx;." \
9   --add-data "config.json;." \
10  app.py
  
```

On the Windows Command Prompt, the command can be placed on one line or saved in a batch file:

Listing 4.19: Windows build batch file

```

1 @echo off
2 python -m PyInstaller --noconfirm --clean --onefile ^
3   --windowed --name SegmentationApp ^
4   --collect-all onnxruntime ^
5   --add-data "model.onnx;." ^
6   --add-data "config.json;." app.py
  
```

The executable is created at:

Listing 4.20: Windows build output

```

1 dist\SegmentationApp.exe
  
```

The `-windowed` option suppresses the terminal window. Omit this option during early testing if terminal messages are useful for diagnosing errors.

4.8.2 Linux Build

Linux uses a colon instead of a semicolon in the `-add-data` argument:

Listing 4.21: Building a standalone Linux application

```
1 python -m PyInstaller \
2   --noconfirm \
3   --clean \
4   --onefile \
5   --windowed \
6   --name SegmentationApp \
7   --collect-all onnxruntime \
8   --add-data "model.onnx:" \
9   --add-data "config.json:" \
10  app.py
```

The Linux executable is created at:

Listing 4.22: Linux build output

```
1 dist/SegmentationApp
```

4.8.3 Operating-System Compatibility

A Windows executable should normally be built on Windows, and a Linux executable should normally be built on Linux. Separate builds may also be required for x86-64 and ARM processors. A one-file application can be large because it contains the interpreter, ONNX Runtime, Pygame, NumPy, Pillow, and other support files.

4.9 CPU and GPU Deployment

CPU inference is preferable for the first portable build because it has fewer external requirements. The example explicitly selects:

Listing 4.23: Selecting CPU inference

```
1 providers=["CPUExecutionProvider"]
```

GPU inference can improve performance, particularly for large segmentation models, but the target computer must have a compatible GPU driver and the libraries required by the selected ONNX Runtime execution provider. The application must also be packaged and tested against those versions.

4.10 Testing the Final Application

Testing should cover more than whether the program opens. A useful validation sequence is:

1. Run the PyTorch model and ONNX model on the same numerical input.
2. Confirm that their logits or predicted labels agree within a reasonable numerical tolerance.
3. Test the unpackaged Pygame application.
4. Build the executable and run it on the development computer.

Consideration	CPU build	GPU build
Portability	Usually easier to distribute	Depends on GPU model, driver, and runtime versions
Setup on target PC	Minimal	Additional driver and acceleration requirements
Executable size	Large	Often larger or accompanied by more libraries
Inference speed	Suitable for small and moderate models	Often faster for large models and high-resolution images
Recommended first step	Yes	Add after the CPU workflow is verified

Table 4.4: Practical comparison of CPU and GPU deployment.

5. Copy the executable to a clean target computer.
6. Test small, large, grayscale, RGB, and unexpected image formats.
7. Confirm that saved masks preserve the intended class colors and image dimensions.
8. Record inference time and memory use on the target hardware.

4.10.1 Common Problems

Problem	Likely cause	Recommended check
ONNX export fails	Unsupported operation or data-dependent model behavior	Inspect the export message and isolate the unsupported layer
Predictions differ from PyTorch	Preprocessing, model mode, or output mismatch	Use one identical tensor and compare intermediate assumptions
Colors are assigned to wrong objects	Class order differs from training	Check the index-to-class mapping and color table
GUI freezes during inference	Inference runs inside the event loop	Move inference to a worker thread or process
Executable cannot find the model	Incorrect bundled-resource path	Use <code>sys._MEIPASS</code> through a resource helper
Executable works only on build PC	Missing native dependency or incompatible architecture	Test on a clean computer with the same operating system and processor architecture
GPU inference does not start	Driver, CUDA, or runtime version mismatch	Check the execution-provider requirements and available providers

Table 4.5: Common deployment problems and initial diagnostic checks.

4.11 Adapting the Example to a Trained Model

To replace the demonstration model with a real model:

1. Copy the exact model architecture used during training into `model_definition.py`.
2. Set the correct number of classes.
3. Load the best checkpoint using the format produced by the training script.
4. Make the model's forward method return the required logits tensor directly.
5. Set the correct input height and width.
6. Copy the training normalization values to `config.json`.
7. Enter class names and mask colors in their exact training order.
8. Export the model and compare PyTorch and ONNX outputs.
9. Run the unpackaged GUI and inspect several predictions.
10. Build and test the standalone application.

For semantic segmentation, the most common output is a tensor with shape $[N, C, H, W]$, where N is the batch size, C is the number of classes, and H and W are the output height and width. Applying `argmax` along the class dimension produces the label map. Classification and object-detection models require different postprocessing.

4.12 Summary

A model trained in PyTorch can be deployed on another computer without a separate PyTorch installation. The trained model is first exported to ONNX and then executed by ONNX Runtime. Pygame can provide a desktop interface for loading images, running inference, displaying predictions, reporting inference time, and saving results. PyInstaller can bundle the GUI, Python interpreter, ONNX Runtime, model, configuration, and supporting libraries into a standalone application.

The most reliable implementation order is to verify the exported ONNX model, test the unpackaged GUI, build a CPU executable, and test it on a clean target computer. GPU packaging should be treated as a separate deployment stage because it introduces additional hardware and software compatibility requirements.

Chapter 5

Publishing an ML Demo with GitHub Pages and an On-Demand Inference API

5.1 Introduction

A trained machine-learning model can be made available through a public website. Visitors can select an image, press a button, and receive a prediction without installing Python, PyTorch, or any desktop software. A practical design separates the application into two parts:

1. a static frontend hosted on GitHub Pages; and
2. an inference API hosted on a server that can run Python and the model.

The GitHub Pages site contains HTML, CSS, and JavaScript. It can display buttons, image previews, progress messages, prediction masks, and download controls. GitHub Pages does not run a persistent Python process, so the model must either run in the visitor's browser or on a separate inference server. This chapter focuses on the second option.

The browser contacts the API only after the visitor requests inference. Normal page views, image previews, and interface interactions do not consume inference compute. This design is useful for models that are too large for browser inference, require a GPU, contain operators that are unavailable in a browser, or should not be distributed to visitors.

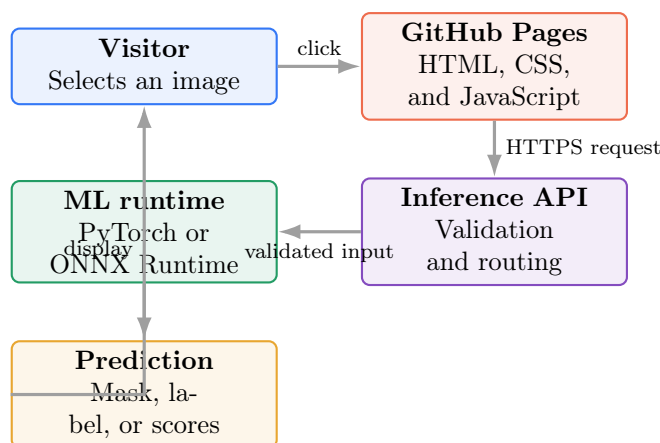


Figure 5.1: Separation between the static GitHub Pages interface and the external inference service.

5.2 Understanding What Loads and When

The phrase *load only when inference is required* can refer to three different operations. They should be distinguished because they affect user experience, cost, and response time.

Operation	When it occurs	Recommended behavior
Load the demo web-page	When the visitor opens the demo page	Load only the interface assets; do not send the image or request inference
Send an inference request	When the visitor clicks the inference button	Upload the selected image and wait for the prediction
Load the model into server memory	When the API process starts or wakes from inactivity	Load once per server process and reuse it for later requests

Table 5.1: Three separate loading events in an API-based ML website.

The frontend should send no inference request until the user presses the button. On the server, however, loading model weights for every request is usually inefficient. The normal pattern is:

1. start the API process;
2. construct the model and load the trained weights once;
3. keep the initialized model in memory;
4. reuse it for each incoming request; and
5. release the model when the hosting platform stops the process.

Some hosting platforms stop inactive services to reduce cost. The first request after such a shutdown may need to start a new process and reload the model. This delay is called a cold start. Later requests handled by the same process are warm requests and are normally faster.

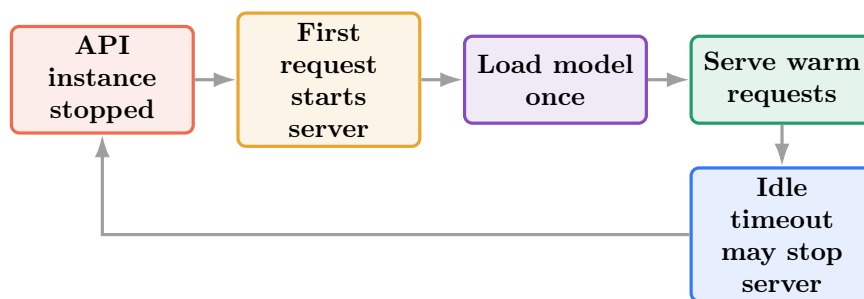


Figure 5.2: Typical lifecycle of a scale-to-zero inference service.

5.3 Browser Inference Versus API Inference

An ONNX model may also run directly in the browser through ONNX Runtime Web. That approach avoids an inference server but requires the visitor to download the model. Table 5.2 summarizes the main differences.

Consideration	Browser inference	External API inference
Model location	Downloaded to the visitor's device	Stored on the server
Python server	Not required	Required for a Python API
Model privacy	Model can be downloaded or inspected	Model remains on the server
Visitor hardware	Determines inference speed	Server hardware determines inference speed
GPU control	Depends on browser and device support	Controlled by the API operator
Operating cost	Mainly static hosting and bandwidth	Compute, storage, and network costs may apply
Image privacy	Image can remain in the browser	Image is transmitted to the API
Large-model suitability	Limited by download time and browser memory	Better when adequate server resources are available
Offline use	Possible after assets are cached	Not available without network access

Table 5.2: Comparison of browser-based and API-based inference.

5.4 End-to-End Request Sequence

The request begins only after the visitor selects an image and confirms the action. The interface should disable the inference button while a request is in progress so that one click does not create several simultaneous requests.

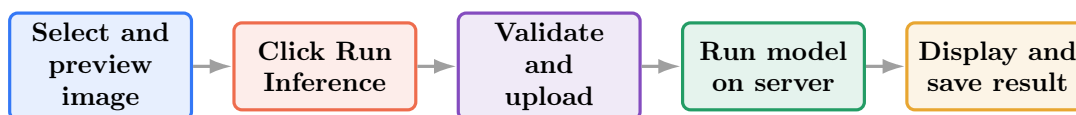


Figure 5.3: Actions performed for one user-initiated inference request.

The server should validate the request before allocating significant model compute. Validation should confirm the file size, media type, decoded image dimensions, and any required authentication or rate-limit policy.

5.5 Recommended Project Organization

The frontend and backend may be kept in separate repositories or in separate directories of one repository. Separate repositories are convenient when GitHub Pages and the API have independent deployment processes.

Listing 5.1: Example frontend and backend organization

```
1 ml-web-demo/  
2 |-- frontend/  
3 | |-- index.html  
4 | |-- style.css  
5 | +-- app.js  
6 +-- backend/  
7     |-- main.py  
8     |-- inference.py  
9     |-- model.onnx  
10    |-- requirements.txt  
11    +-- Dockerfile
```

Only the contents of **frontend/** are published through GitHub Pages. The **backend/** directory is deployed to a service that can run a persistent or serverless Python process.

5.6 Building the GitHub Pages Frontend

5.6.1 HTML Interface

The interface needs a file input, image preview, inference button, status message, and result element. Listing 5.2 provides a compact starting point.

Listing 5.2: HTML interface for an image inference demo

```

1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <meta name="viewport"
6      content="width=device-width, initial-scale=1">
7    <title>Semantic Segmentation Demo</title>
8    <link rel="stylesheet" href="style.css">
9  </head>
10 <body>
11   <main class="container">
12     <h1>Semantic Segmentation Demo</h1>
13     <p>Select an image and run the trained model.</p>
14
15     <input id="imageInput" type="file"
16       accept="image/png,image/jpeg">
17     <button id="runButton" disabled>
18       Run Inference
19     </button>
20
21     <p id="status" role="status">
22       Select an image to begin.
23     </p>
24
25     <section class="panels">
26       <figure>
27         <figcaption>Input image</figcaption>
28         <img id="preview" alt="Selected input">
29       </figure>
30       <figure>
31         <figcaption>Prediction</figcaption>
32         <img id="result" alt="Model prediction">
33       </figure>
34     </section>
35
36     <a id="downloadLink" hidden
37       download="prediction.png">
38       Download result
39     </a>
40   </main>
41   <script src="app.js"></script>
42 </body>
43 </html>

```

The `accept` attribute filters the file-selection dialog, but it is not a security control. The API must independently inspect and decode every uploaded file.

5.6.2 Basic Styling

The layout in Listing 5.3 adapts from two columns on a wide screen to one column on a narrow screen.

Listing 5.3: CSS for the inference interface

```
1 :root {
2   color-scheme: light dark;
3   font-family: Arial, sans-serif;
4 }
5
6 body {
7   margin: 0;
8   background: #111827;
9   color: #f3f4f6;
10 }
11
12 .container {
13   width: min(1100px, 92vw);
14   margin: 2rem auto;
15 }
16
17 button {
18   margin-left: 0.7rem;
19   padding: 0.65rem 1rem;
20   border: 0;
21   border-radius: 0.45rem;
22   background: #3478f6;
23   color: white;
24   font-weight: 700;
25   cursor: pointer;
26 }
27
28 button:disabled {
29   background: #6b7280;
30   cursor: not-allowed;
31 }
32
33 .panels {
34   display: grid;
35   grid-template-columns: 1fr 1fr;
36   gap: 1rem;
37 }
38
39 figure {
40   margin: 0;
41   padding: 1rem;
42   background: #1f2937;
43   border-radius: 0.7rem;
44 }
45
46 img {
47   display: block;
48   width: 100%;
49   min-height: 220px;
50   object-fit: contain;
51 }
52
53 @media (max-width: 750px) {
54   .panels {
55     grid-template-columns: 1fr;
56   }
57 }
```

5.6.3 Previewing the Image Without Calling the API

The selected image can be previewed locally with an object URL. This operation does not upload the image and does not call the model.

Listing 5.4: Local image preview without inference

```
1  const imageInput = document.querySelector(  
2    "#imageInput"  
3  );  
4  const runButton = document.querySelector(  
5    "#runButton"  
6  );  
7  const preview = document.querySelector("#preview");  
8  const result = document.querySelector("#result");  
9  const statusText = document.querySelector("#status");  
10 const downloadLink = document.querySelector(  
11   "#downloadLink"  
12 );  
13  
14 let previewUrl = null;  
15 let resultUrl = null;  
16  
17 imageInput.addEventListener("change", () => {  
18   const file = imageInput.files[0];  
19  
20   if (previewUrl) {  
21     URL.revokeObjectURL(previewUrl);  
22   }  
23  
24   result.removeAttribute("src");  
25   downloadLink.hidden = true;  
26  
27   if (!file) {  
28     preview.removeAttribute("src");  
29     runButton.disabled = true;  
30     statusText.textContent =  
31       "Select an image to begin.";  
32     return;  
33   }  
34  
35   previewUrl = URL.createObjectURL(file);  
36   preview.src = previewUrl;  
37   runButton.disabled = false;  
38   statusText.textContent =  
39     "Image ready. Click Run Inference.";  
40 });
```

5.6.4 Calling the API Only After a Button Click

The API address must use HTTPS when the GitHub Pages site uses HTTPS. Browsers normally block an HTTPS webpage from calling an insecure HTTP endpoint.

Listing 5.5: Sending the inference request

```
1  const API_URL =  
2    "https://api.example.com/predict";  
3  
4  
5  async function requestPrediction(file) {  
6    const formData = new FormData();  
7    formData.append("image", file);  
8  
9    const response = await fetch(API_URL, {  
10      method: "POST",  
11      body: formData  
12    });  
13  
14    if (!response.ok) {  
15      let message = "Inference request failed.";  
16  
17      try {  
18        const errorData = await response.json();  
19        message = errorData.detail || message;  
20      } catch {  
21        // Keep the default message.  
22      }  
23  
24      throw new Error(message);  
25    }  
26  
27    return {  
28      imageBlob: await response.blob(),  
29      inferenceTime:  
30        response.headers.get("X-Inference-Time-Ms")  
31    };  
32  }
```

Listing 5.6: Connecting inference to the Run button

```
1 runButton.addEventListener("click", async () => {
2   const file = imageInput.files[0];
3   if (!file) {
4     return;
5   }
6
7   runButton.disabled = true;
8   statusText.textContent = "Running inference...";
9
10  try {
11    const prediction = await requestPrediction(file);
12
13    if (resultUrl) {
14      URL.revokeObjectURL(resultUrl);
15    }
16
17    resultUrl = URL.createObjectURL(
18      prediction.imageBlob
19    );
20    result.src = resultUrl;
21    downloadLink.href = resultUrl;
22    downloadLink.hidden = false;
23
24    const time = prediction.inferenceTime;
25    statusText.textContent = time
26      ? "Completed in " + time + " ms."
27      : "Inference completed.";
28  } catch (error) {
29    statusText.textContent = error.message;
30  } finally {
31    runButton.disabled = false;
32  }
33 });
```

This frontend expects the API to return a PNG image. A classification API may return JSON containing the predicted label and confidence values instead.

5.6.5 Adding the Demo as a Separate Page

The inference interface does not need to be the homepage. It may be stored as `demo.html` and linked from the main site:

Listing 5.7: Linking to a separate ML demo page

```
1 <a href="/demo.html">Open the ML Demo</a>
```

In that arrangement, the demo's JavaScript loads when the visitor opens the demo page, while the inference API is still contacted only after the visitor presses the inference button.

5.7 Building the FastAPI Inference Service

FastAPI provides request parsing, validation, asynchronous file handling, automatic API documentation, and standard HTTP responses. The model may be executed with PyTorch or ONNX Runtime. The following example uses ONNX Runtime because the model is already available as `model.onnx`.

5.7.1 Loading the Model Once

Listing 5.8 creates one inference session when the Python module is imported. Requests handled by that process reuse the same session.

Listing 5.8: Server-side model initialization

```
1 import io
2 import time
3
4 import numpy as np
5 import onnxruntime as ort
6 from PIL import Image, UnidentifiedImageError
7
8
9 INPUT_WIDTH = 512
10 INPUT_HEIGHT = 192
11 MEAN = np.asarray(
12     [0.485, 0.456, 0.406],
13     dtype=np.float32
14 ).reshape(1, 1, 3)
15 STD = np.asarray(
16     [0.229, 0.224, 0.225],
17     dtype=np.float32
18 ).reshape(1, 1, 3)
19 COLORS = np.asarray([
20     [128, 64, 128],
21     [244, 35, 232],
22     [70, 130, 180],
23     [107, 142, 35],
24 ], dtype=np.uint8)
25
26 SESSION = ort.InferenceSession(
27     "model.onnx",
28     providers=["CPUExecutionProvider"],
29 )
30 INPUT_NAME = SESSION.get_inputs()[0].name
```

For GPU deployment, the available execution providers and their system requirements must be checked before selecting a GPU provider. The API should fail clearly during startup if the required provider is unavailable rather than silently using an unintended device.

5.7.2 Preprocessing and Prediction

The preprocessing must exactly match training. Listing 5.9 decodes the uploaded bytes, creates the input tensor, executes the model, and creates a colored overlay.

Listing 5.9: ONNX inference and segmentation overlay

```

1  def run_model(image_bytes):
2      try:
3          original = Image.open(
4              io.BytesIO(image_bytes)
5          ).convert("RGB")
6      except UnidentifiedImageError as error:
7          raise ValueError(
8              "The uploaded file is not a valid image."
9          ) from error
10
11     resized = original.resize(
12         (INPUT_WIDTH, INPUT_HEIGHT),
13         Image.Resampling.BILINEAR,
14     )
15     array = np.asarray(
16         resized, dtype=np.float32
17     ) / 255.0
18     array = (array - MEAN) / STD
19     tensor = np.transpose(
20         array, (2, 0, 1)
21     )[None, ...].astype(np.float32)
22
23     started = time.perf_counter()
24     output = SESSION.run(
25         None, {INPUT_NAME: tensor}
26     )[0]
27     elapsed_ms = (
28         time.perf_counter() - started
29     ) * 1000.0
30
31     mask = np.argmax(output[0], axis=0)
32     color_mask = Image.fromarray(COLORS[mask])
33     color_mask = color_mask.resize(
34         original.size,
35         Image.Resampling.NEAREST,
36     )
37     overlay = Image.blend(
38         original, color_mask, alpha=0.45
39     )
40
41     buffer = io.BytesIO()
42     overlay.save(buffer, format="PNG")
43     return buffer.getvalue(), elapsed_ms

```

This example measures only model execution time. Total user-perceived latency also includes upload time, request scheduling, decoding, preprocessing, postprocessing, PNG encoding, download time, and browser rendering.

5.7.3 Creating the API Application

The API accepts PNG and JPEG uploads, limits the number of bytes read, runs the model, and returns the overlay as a PNG response.

Listing 5.10: FastAPI application and CORS configuration

```
1 from fastapi import FastAPI, File, HTTPException
2 from fastapi import UploadFile
3 from fastapi.middleware.cors import CORSMiddleware
4 from fastapi.responses import Response
5
6 from inference import run_model
7
8
9 app = FastAPI(title="Segmentation API")
10
11 app.add_middleware(
12     CORSMiddleware,
13     allow_origins=[
14         "https://shahriar88.github.io"
15     ],
16     allow_credentials=False,
17     allow_methods=["POST", "GET"],
18     allow_headers=["Content-Type"],
19     expose_headers=["X-Inference-Time-Ms"],
20 )
21
22 MAX_UPLOAD_BYTES = 10 * 1024 * 1024
23 ALLOWED_TYPES = {"image/jpeg", "image/png"}
24
25
26 @app.get("/health")
27 def health():
28     return {"status": "ready"}
```

Listing 5.11: Validated prediction endpoint

```

1 @app.post("/predict")
2 async def predict(
3     image: UploadFile = File(...)
4 ):
5     if image.content_type not in ALLOWED_TYPES:
6         raise HTTPException(
7             status_code=415,
8             detail="Upload a PNG or JPEG image.",
9         )
10
11     data = await image.read(
12         MAX_UPLOAD_BYTES + 1
13     )
14     if len(data) > MAX_UPLOAD_BYTES:
15         raise HTTPException(
16             status_code=413,
17             detail="Image exceeds the 10 MB limit.",
18         )
19
20     try:
21         output_png, elapsed_ms = run_model(data)
22     except ValueError as error:
23         raise HTTPException(
24             status_code=400,
25             detail=str(error),
26         ) from error
27
28     return Response(
29         content=output_png,
30         media_type="image/png",
31         headers={
32             "X-Inference-Time-Ms":
33                 f"{elapsed_ms:.1f}"
34         },
35     )

```

The media type sent by the browser is useful for early rejection but cannot be trusted by itself. Decoding the bytes as an image provides an additional check. Production systems should also enforce maximum pixel dimensions to prevent a small compressed file from expanding into an unexpectedly large image in memory.

5.7.4 Response Formats

The response format should match the task and the information the interface must display.

5.8 Running the API Locally

Create and activate a virtual environment, then install the required packages:

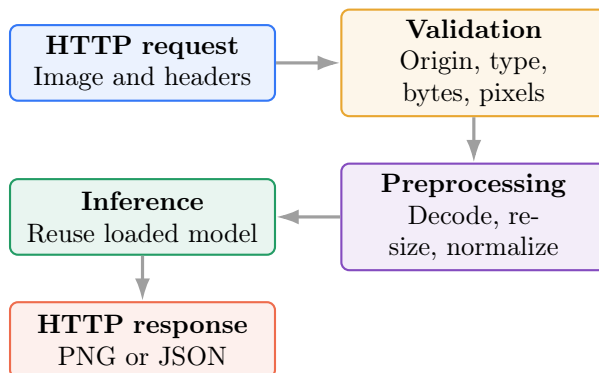


Figure 5.4: Validation and processing stages inside the inference API.

Format	Suitable tasks	Notes
JSON	Classification, regression, detection meta-data	Easy to inspect and display; coordinates and scores can be returned as arrays
PNG image	Segmentation masks, heatmaps, and overlays	Compact for visual results and easy to display with an image element
JSON with Base64 image	Metadata and image in one response	Convenient but larger than returning binary image data directly
Binary array	Scientific or high-volume numeric output	Efficient but requires a documented encoding and JavaScript decoder

Table 5.3: Possible API response formats for common ML tasks.

Listing 5.12: Backend requirements.txt

```
1 fastapi
2 uvicorn[standard]
3 python-multipart
4 onnxruntime
5 numpy
6 pillow
```

Listing 5.13: Installing and starting the local API

```
1 python -m venv .venv
2 source .venv/bin/activate
3 python -m pip install -r requirements.txt
4 uvicorn main:app --host 127.0.0.1 --port 8000
```

On Windows, activate the environment with:

Listing 5.14: Activating the backend environment on Windows

```
1 .venv\Scripts\activate
```

The health endpoint can be checked from a second terminal:

Listing 5.15: Checking the API health endpoint

```
1 curl http://127.0.0.1:8000/health
```

The prediction endpoint can be tested independently of the website:

Listing 5.16: Testing the prediction endpoint with curl

```
1 curl -X POST \
2   -F "image=@test_image.jpg" \
3   http://127.0.0.1:8000/predict \
4   --output prediction.png
```

Local testing should be completed before the frontend URL is changed to the public API address.

5.9 Containerizing the API

A container provides a reproducible Python environment and is supported by many CPU and GPU hosting platforms. The following Dockerfile creates a basic CPU deployment:

Listing 5.17: Dockerfile for the inference API

```
1 FROM python:3.12-slim
2
3 ENV PYTHONDONTWRITEBYTECODE=1
4 ENV PYTHONUNBUFFERED=1
5
6 WORKDIR /app
7
8 COPY requirements.txt .
9 RUN python -m pip install \
10     --no-cache-dir \
11     -r requirements.txt
12
13 COPY main.py inference.py model.onnx ./
14
15 EXPOSE 8000
16
17 CMD ["uvicorn", "main:app",
18     "--host", "0.0.0.0",
19     "--port", "8000",
20     "--workers", "1"]
```

Build and test the image locally:

Listing 5.18: Building and running the API container

```
1 docker build -t segmentation-api .
2 docker run --rm -p 8000:8000 segmentation-api
```

Each worker process normally loads its own model and therefore consumes its own memory. Increasing the worker count without measuring memory use can exhaust RAM or GPU memory. For a GPU service, concurrency is often controlled at the request or container level instead of creating many model copies.

5.10 Choosing an API Hosting Pattern

The backend cannot run on GitHub Pages. It must be deployed to a platform that supports Python processes, containers, functions, or GPU workloads.

The correct choice should be based on measured model size, initialization time, inference time, memory use, traffic, and acceptable first-request delay. A small CPU model may perform well on a modest container, while a large segmentation model may require a GPU and stricter concurrency controls.

5.11 Configuring CORS Correctly

Cross-Origin Resource Sharing controls which webpages may read responses from the API through a browser. The frontend and API normally have different origins, for example:

Listing 5.19: Example frontend and API origins

```
1 Frontend: https://shahriar88.github.io
2 API: https://api.example.com
```

Hosting pattern	Advantages	Limitations	Suitable use
Always-on CPU service	Predictable availability and simple model reuse	Continues consuming resources while idle	Small and moderate models with regular traffic
Scale-to-zero container	Lower idle cost and automatic scaling	Cold-start delay and platform time limits	Public demonstrations with occasional use
Serverless function	Simple request-based operation	Package, memory, runtime, and execution-duration limits	Small models with short inference time
Always-on GPU service	Low warm-request latency for large models	Higher continuous cost	Frequent or latency-sensitive inference
On-demand GPU service	GPU is allocated when needed	Longer startup delay and more complex lifecycle	Large models with infrequent requests

Table 5.4: General hosting patterns for an external inference API.

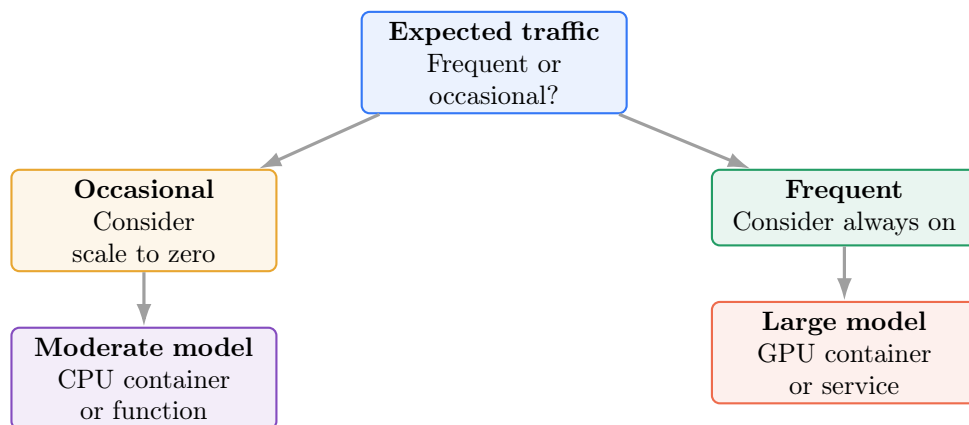


Figure 5.5: Simplified hosting decision based on traffic and model requirements.

The API must explicitly allow the frontend origin. During local development, the local frontend origin may also be included:

Listing 5.20: Allowing production and local frontend origins

```
1 ALLOWED_ORIGINS = [
2     "https://shahriar88.github.io",
3     "http://127.0.0.1:5500",
4     "http://localhost:5500",
5 ]
```

CORS is a browser access policy, not authentication. It does not prevent a script, command-line client, or another server from sending requests directly to a public API. Rate limits, quotas, authentication, and infrastructure rules must be enforced by the backend or an API gateway.

5.12 Security, Privacy, and Abuse Prevention

A public inference endpoint should be treated as an internet-facing service. At minimum, the design should address:

- maximum upload size and decoded image dimensions;
- allowed request methods and media types;
- request and inference timeouts;
- per-client rate limits and overall quotas;
- temporary-file cleanup if files are written to disk;
- safe error messages that do not reveal server paths or configuration;
- dependency updates and container vulnerability checks;
- HTTPS for all production traffic;
- logging that avoids retaining sensitive image data; and
- a clear notice explaining how uploaded images are processed.

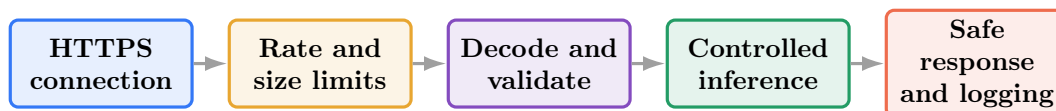


Figure 5.6: Layered controls for a public inference endpoint.

5.12.1 Do Not Put a Secret API Key in Public JavaScript

All JavaScript delivered through a public GitHub Pages site can be inspected by visitors. A permanent secret embedded in `app.js` is therefore not secret. If the demo is public, the API should be designed for public access with strict rate limits and usage caps. If access must be restricted, use a real authentication flow that issues short-lived user-specific credentials or place a trusted backend between the public page and the protected inference service.

5.12.2 Uploaded Image Privacy

With API inference, the selected image leaves the visitor's device. The site should state whether images are stored, logged, or discarded after inference. If the application does not require retention, process images in memory and avoid saving request bodies. Infrastructure logs and error-reporting tools should also be reviewed because they may retain request metadata.

5.13 Managing Latency and User Experience

The interface should communicate each stage of a request. At a minimum, it should show:

- image selected;
- upload in progress;
- inference in progress;
- prediction completed;
- invalid input;
- service unavailable; and
- request timed out.

The total latency observed by the visitor can be represented as:

$$T_{\text{total}} = T_{\text{upload}} + T_{\text{queue}} + T_{\text{decode}} + T_{\text{preprocess}} + T_{\text{model}} + T_{\text{postprocess}} + T_{\text{download}}.$$

The `X-Inference-Time-Ms` header in the example reports only T_{model} . Both model time and total browser time may be measured when diagnosing performance.

5.13.1 Preventing Duplicate Requests

Disable the inference button until the current request finishes. For a more advanced interface, use an `AbortController` so the visitor can cancel a request:

Listing 5.21: Adding request cancellation

```
1 let activeController = null;
2
3 function cancelActiveRequest() {
4   if (activeController) {
5     activeController.abort();
6     activeController = null;
7   }
8 }
9
10 async function requestPrediction(file) {
11   cancelActiveRequest();
12   activeController = new AbortController();
13
14   const formData = new FormData();
15   formData.append("image", file);
16
17   const response = await fetch(API_URL, {
18     method: "POST",
19     body: formData,
20     signal: activeController.signal
21   });
22
23   activeController = null;
24   return response;
25 }
```

Canceling a browser request does not always stop inference that has already started on the server. Server-side timeouts and cancellation behavior must be handled separately.

5.14 Scaling and Concurrency

Multiple visitors may request inference at the same time. The service must define whether requests are processed concurrently, queued, batched, or rejected when capacity is reached.

For an initial public demonstration, a small queue, a strict concurrency limit, and a clear busy response are safer than allowing an unlimited number of simultaneous predictions.

5.15 Versioning the Model and API

The webpage and API should expose enough version information to reproduce a prediction. A response may include headers or JSON fields for:

- API version;
- model name and version;
- class-map version;
- input preprocessing version; and
- inference runtime version.

Strategy	Benefit	Tradeoff
Single-request worker	Simple and predictable memory use	Later requests wait
Multiple process workers	Uses multiple CPU cores	Each process may load another model copy
Threaded requests	Can overlap input/output operations	Model runtime and shared-state safety must be verified
Request queue	Protects the model from overload	Adds waiting time and requires queue limits
Dynamic replicas	Adds capacity during traffic increases	Startup delays and higher cost are possible
Batching	Can improve accelerator utilization	Adds scheduling delay and implementation complexity

Table 5.5: Common concurrency strategies for an inference service.

A versioned endpoint such as `/v1/predict` allows a later `/v2/predict` endpoint to introduce a different response format without immediately breaking the existing webpage.

5.16 Testing Before Public Release

Testing should be performed at several levels.

1. **Model test:** compare server predictions with the validated local PyTorch or ONNX predictions.
2. **Preprocessing test:** confirm image resizing, normalization, channel order, and class order.
3. **Endpoint test:** send valid and invalid requests directly to the API.
4. **CORS test:** verify the production GitHub Pages origin and reject unintended browser origins.
5. **Frontend test:** check preview, loading status, errors, result display, and download behavior.
6. **Clean-browser test:** test without cached assets or an existing authenticated session.
7. **Mobile test:** check narrow layouts, camera images, and memory use.
8. **Load test:** measure behavior under the expected number of simultaneous users.
9. **Cold-start test:** measure the first request after the service has been inactive.

5.16.1 Common Problems

5.17 Deployment Checklist

1. Export and validate the trained model.
2. Implement the inference function and test it locally.

Symptom	Likely cause	Initial check
Browser reports a CORS error	Frontend origin is not allowed or preflight failed	Compare the exact scheme, domain, and port with the CORS list
Mixed-content error	HTTPS page calls an HTTP API	Deploy the API with HTTPS
First request is very slow	Service is starting and loading the model	Measure cold-start and model initialization time
Every request is slow	Model is loaded inside the endpoint	Move initialization outside the request function
HTTP 413 response	Upload exceeds the configured byte limit	Resize or compress the image, or revise the documented limit
HTTP 415 response	Unsupported media type	Upload a permitted PNG or JPEG
Wrong segmentation colors	Class order or preprocessing differs from training	Compare configuration and a known local prediction
API memory failure	Image dimensions, worker count, or model size is too large	Limit decoded pixels and measure memory per worker
Repeated charges or overload	Public endpoint has no effective usage control	Add rate limits, quotas, monitoring, and concurrency limits

Table 5.6: Common frontend and inference API problems.

3. Add file-type, byte-size, and decoded-dimension limits.
4. Create the `/health` and `/predict` endpoints.
5. Configure CORS for the exact GitHub Pages origin.
6. Test the API independently with `curl`.
7. Build the HTML, CSS, and JavaScript interface.
8. Confirm that image preview does not contact the API.
9. Confirm that only the inference button sends the request.
10. Deploy the API with HTTPS.
11. Replace the development API URL in `app.js`.
12. Publish the frontend through GitHub Pages.
13. Test warm requests, cold starts, invalid files, and timeouts.
14. Add monitoring, rate limits, concurrency limits, and cost alerts.
15. Publish a short image-processing and privacy notice.

5.18 Summary

A GitHub Pages site can provide a complete user interface for a machine-learning demo while an external API performs inference. The static page previews the selected image locally and contacts the API only when the visitor presses the inference button. The server should normally load the model once per process and reuse it, rather than loading the weights for every image.

This architecture keeps large model files away from the browser, allows the use of controlled CPU or GPU hardware, and permits the model to be updated without republishing the frontend. It also introduces responsibilities that do not exist in a fully browser-based application, including compute cost, cold starts, CORS configuration, upload validation, privacy disclosure, rate limiting, concurrency control, and service monitoring.

The most reliable development order is to validate the model locally, test the API independently, connect the frontend, deploy the API with HTTPS, and then publish the GitHub Pages interface. A public endpoint should be released only after its resource limits and abuse controls have been tested.

Index

- cpus-per-task, [9](#)
- gres, [9](#)
- time, [10](#)
- N, [9](#)
- p, [9](#)
- #SBATCH directives, [7](#)
- cold start, [76](#)
- Conda environment
 - IR, [20](#)
- CORS, [90](#)
- CRADLE, [7](#)
- CUDA, [10](#)
- cuDNN, [10](#)
- environment
 - shared, [8](#)
- FastAPI, [84](#)
- GitHub Pages, [75](#)
- GPU
 - partition, [9](#)
 - request, [9](#)
- GPU workload, [7](#)
- GRES, [9](#)
- HPC cluster, [1](#)
- machine learning
 - deployment, [59](#)
 - web deployment, [75](#)
- Micromamba, [20](#)
- node
 - compute, [9](#)
- ONNX, [59](#)
- ONNX Runtime, [59](#)
- pandas, [16](#)
- partition, [9](#)
- PyInstaller, [59](#)
- Python environment, [17](#)
- PyTorch, [59](#)
- sacct, [26](#)
- scheduler
 - directives, [8](#)
- scp, [13](#), [20](#)
- SLURM, [1](#)
 - batch script, [7](#)
 - environment variables, [25](#)
 - resources, [25](#)
- ssh, [7](#)
- TensorFlow, [7](#)
 - shared installation, [10](#)
- time limit, [10](#)
- university cluster, [7](#)
- venv, [17](#)
- virtual environment, [7](#)
- virtualenv-clone, [15](#)
- VPN, [7](#)